

七、图像采集与数据集准备实验指导

1. 实验描述

从本次实验开始，我们会接触到人工智能技术应用中的计算机视觉应用，后续实验中主要基于智能机器人的摄像头获取图像，并调用 AI 模型实现图像识别、人脸检测、目标检测等任务。

想要完成后续的视觉任务，需要先进行 AI 模型的训练。AI 模型训练前提是需要做好图像数据集的准备工作，包括图像采集、图像数据标注、数据格式转换、数据集划分等，然后基于准备好的数据集，才能在合适的环境里开展 AI 模型训练、模型评估、模型导出、模型部署等一系列流程，最后在智能机器人中直接加载应用我们训练完成后的推理模型就可以完成后续对应的目标检测等任务。

本节实训主要完成数据集的准备工作，主要包括三项任务，分别是图像数据采集、数据标注以及数据集的划分。图像采集时我们基于智能机器人进行，通过 `python` 指令的形式启动智能机器人的图像采集程序，通过手柄控制机器人，完成图像数据的采集。数据标注阶段，我们主要基于 Easydata 智能平台在线完成数据标注任务，还可参考课程提供的参考文档采用 Labelme 进行数据标注，最后调用本实训提供的数据集划分脚本代码完成数据集的划分任务。

2. 实验目标

- (1) 掌握图像采集的代码逻辑，通过手柄控制智能机器人进行图像采集与保存；
- (2) 掌握在线进行图像数据标注的方法；
- (3) 完成数据划分的任务。

3. 实验步骤

步骤 1：启动智能机器人，打开工程文件

打开智能机器人电源开关后，等待系统自行启动。在机器人终端，参考实训一，通过指令的形式打开编译工具 VSCode，再通过 VSCode 打开智能机器人的工程文件目录。

通过路径/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/找到本实验中图像采集的代码文件 sebotCollection.py。

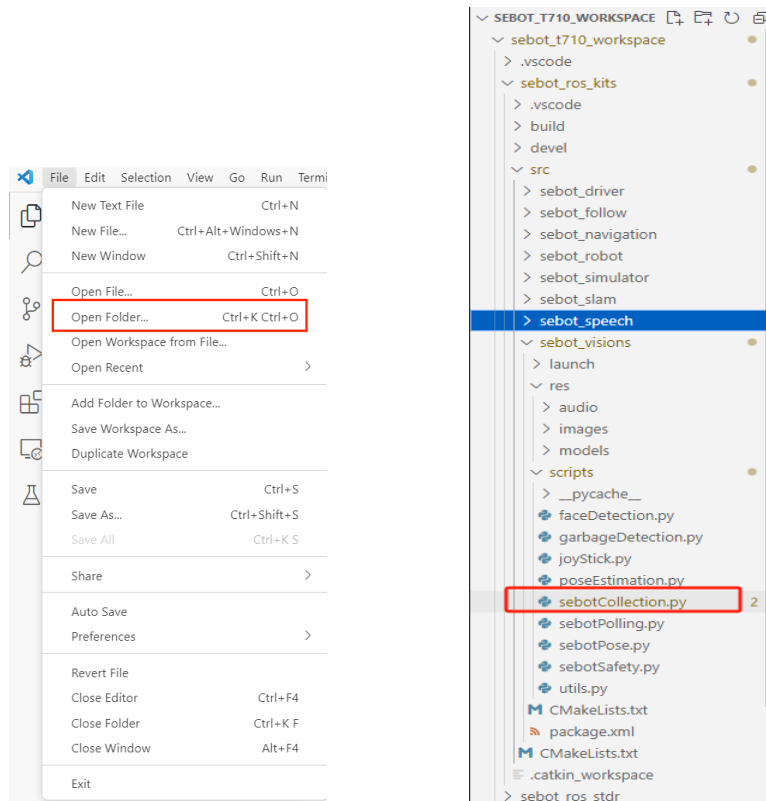


图 7-1 VSCode 打开工程的按钮及打开后的界面

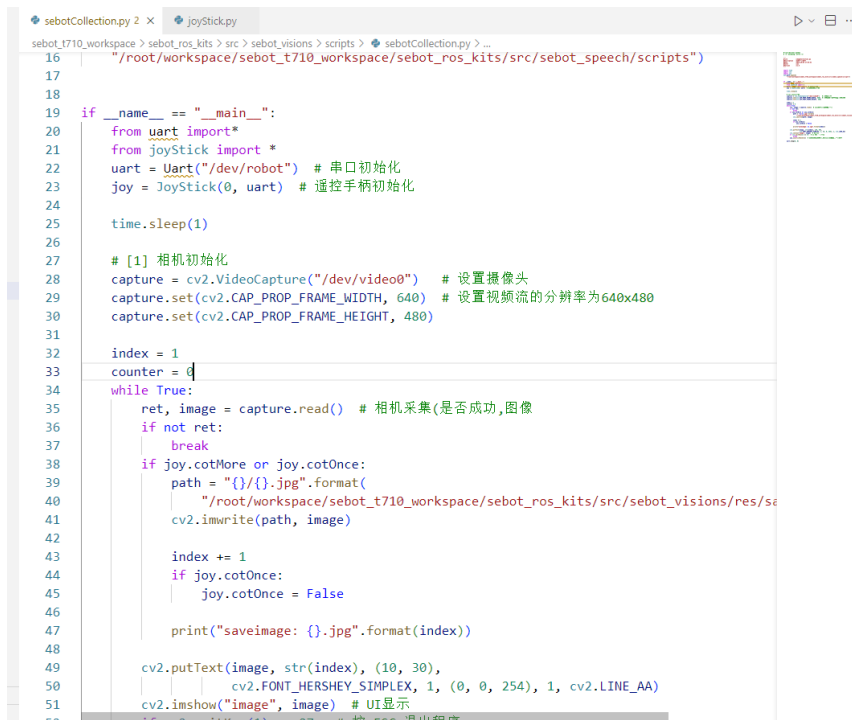


图 7-2 打开 sebotCollection 文件后的界面

步骤 2：学习图像采集的代码逻辑（sebotCollection.py）

在“sebotCollection.py”文件中，图像采集的代码逻辑主要分为如下几个步骤：

第一步，引入函数库；

第二步，初始化，串口、遥控手柄以及相机的初始化；

第三步，遥控手柄控制智能机器人根据不同的方式进行图像采集功能。

（1）引入函数库

1) 导入 time 库：time 模块，用于处理时间相关的操作。

2) 导入 cv2 库：这是 OpenCV 库的 Python 接口，用于图像和视频处理。

3) 导入 sys 库：用于与 Python 解释器进行交互，包括命令行参数等。sys.path.append() 对应的代码功能是将指定的路径添加到 Python 解释器的搜索路径中。在这段代码中，将"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/scripts"路径添加到搜索路径中，这样可以在后续代码中使用该路径下的模块和脚本文件。

```
import time
import cv2
import sys
sys.path.append(

"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech
/scripts")
```

（2）初始化（缺失代码需填空）

这段代码的主要功能是初始化串口与摄像头，并且定义了两个变量 index 和 counter。通过导入 uart 模块和 joyStick 模块，创建了串口和遥控手柄的对象，用于与设备进行通信。接着，通过调用 OpenCV 库提供的函数，初始化了摄像头对象并设置视频流的分辨率。最后，将 index 和 counter 初始化为 1 和 0 用于后续代码使用。

```
if __name__ == "__main__":
    from uart import *
    from joyStick import *
    uart = Uart("/dev/robot") # 串口初始化
    joy = JoyStick(0, uart) # 遥控手柄初始化

    time.sleep(1)

    # [1] 相机初始化
    capture = cv2.VideoCapture("/dev/video0") # 设置摄像头
    # 设置视频流的分辨率为 640x480
```

```
# ##### 代码填空开始处 ##### #
"""
    代码填空提示：代码一共两行
    1. 设置视频界面的长度：通过 capture 调用 set 函数，一共有两个参数
    (cv2.CAP_PROP_FRAME_WIDTH, 640)
    2. 设置视频界面的宽度：通过 capture 调用 set 函数，一共有两个参数
    (cv2.CAP_PROP_FRAME_HEIGHT, 480)
"""
# ##### 代码填空结束处 ##### #
index = 1
counter = 0
```

(3) 遥控手柄控制智能机器人图像采集（缺失代码需填空）

这段代码的功能是循环读取摄像头采集的图像数据，并根据遥控手柄的按键操作，保存图像、显示图像和控制机器人的速度。在循环中，首先判断是否成功读取图像，然后判断遥控手柄是否有按键操作，若有则保存图像、更新图像编号和打印保存信息。接着，在当前图像上添加文本信息和显示图像。如果按下 ESC 键，则退出程序。最后，处理遥控手柄的按键操作和停止串口通信。

```
while True:
    ret, image = capture.read() # 相机采集(是否成功)
    if not ret:
        break
    if joy.cotMore or joy.cotOnce:
        path = "{}/{}.jpg".format(
            "/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_vision
            s/res/sample", index)
        cv2.imwrite(path, image)
        # 控制名称
        # ##### 代码填空开始处 ##### #
        """
            代码填空提示：代码一行
            1. 控制命名的索引/变量值计数+1
        """
        # ##### 代码填空结束处 ##### #
        if joy.cotOnce:
            joy.cotOnce = False

        print("saveimage: {}.jpg".format(index))

    cv2.putText(image, str(index), (10, 30),
                 cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 254), 1,
                 cv2.LINE_AA)
```

```
cv2.imshow("image", image) # UI 显示
if cv2.waitKey(1) == 27: # 按 ESC 退出程序
    break
joy.joystickHandle() # 遥控手柄获取键值,控制机器人速度和图像采集

uart.stop(0, 0)
```

其中 joystickHandle()函数是主要用于遥控手柄的功能函数，主要功能是处理遥控手柄的按键操作，并根据按键操作控制机器人速度和图像采集。函数通过调用串口传输函数发送心跳信号与机器人保持通信连接。在遥控手柄的事件处理循环中，根据按键按下与否及摇杆、方向键盘的运动状态，执行相应的机器人控制动作和更新速度参数。最后通过串口通信发送速度控制指令给机器人，该函数的目的是实现遥控手柄与机器人之间的交互控制。对代码感兴趣的同学可以详细查看该函数的内部代码，智能机器人控制图像采集操作在后面实操部分会进行详细讲解。

步骤 3：手柄控制机器人进行图像采集实操

第一步，在机器人系统中启动图像采集程序。

打开终端，通过 cd 指令切换到对应的文件夹内，然后执行 sebotCollection.py 工程文件。

```
python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotCollection.py
```

图示如下：

```
root@paddlepi:~# python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotCollection.py
```

图 7-3 执行图像数据采集功能

第二步，按照如下图所示的手柄操作进行步骤采集。注意图像采集过程，按钮按下一次，相机拍摄一次，持续按下按钮也只拍摄一次。

可以将后续实验所需的图像进行采集，比如在摄像头的范围内呈现人脸，或者垃圾（纸团、塑料瓶等），遥控机器人进行前进或转向，或低速，或高速行进，进行不同角度的图像采集。



图 7-4 手柄控制

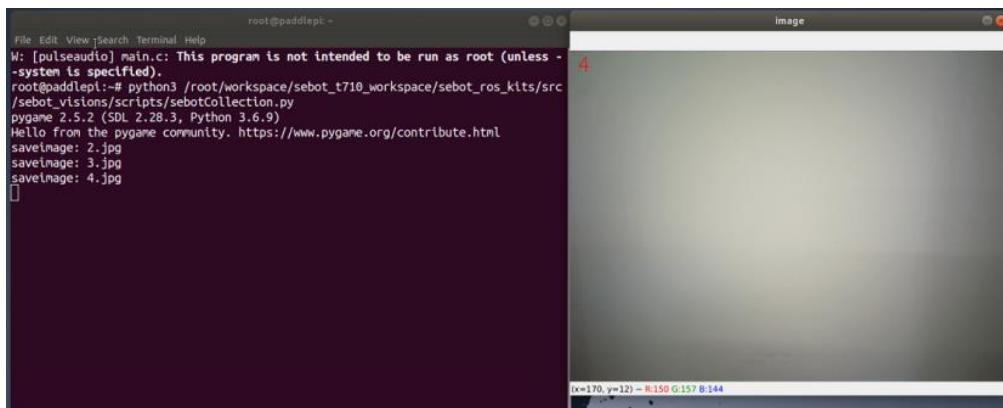


图 7-5 相机拍摄

步骤 4：查看采集的图像

根据步骤 2 的代码逻辑，若手柄有相关按键操作，会自动保存图像。图像保存路径如下：

“workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/images/sample”，在该文件夹中即可查看所有采集到的图像文件。

一般 AI 模型训练需要的图像数据都在几百、上千张甚至数万张，数据量大一些模型训练效果会更好(当然也不是越多越好)。同学们可以操控机器人进行多一些的图像数据采集，比如人脸数据可以采集几百张，或者参考课程提供的数据集进行垃圾图像数据的采集，为后续的图像数据标注提供基本素材。

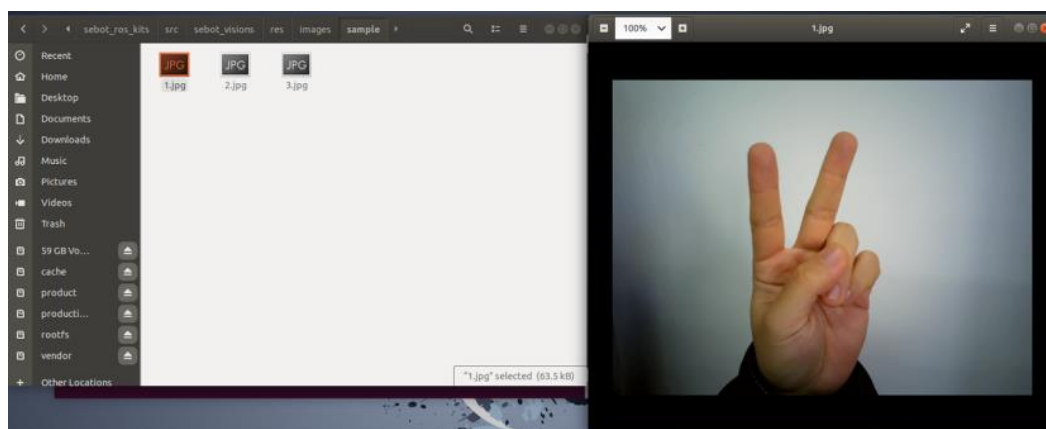


图 7-6 采集完成后图像保存的文件夹

步骤 5：数据标注过程

采集的图像数据必须进行标注后才能便于后续的模式学习与训练，并且模型训练过程中所需要的数据集格式一般均为 VOC2007 格式。数据标注在目标检测中发挥着至关重要的作用，它可以用于训练目标检测模型、提高检测的准确性以及评估模型的性能。标注准确的图片可以有效的提高模型训练的识别率，模型训练是否成功、识别结果能否正确与数据标注都有重要联系。

Labelme 与 EasyData 都是常用于图像标注的工具，它们各自有自身的特点和优势。Labelme 是一个开源工具，相对来说更加简便易用，安装过后不用联网也可以在本地进行数据标注工作；而 Easydata 是一种基于 Web 的图像和数据标注平台，专门用于机器学习和人工智能应用的数据准备和标注任务，在用户界面和交互设计上更加注重用户体验，提供了更加友好和直观的操作方式，只是需要联网使用，是一种线上标注方式。下面我们概要介绍云端的 EasyData 数据标注平台在线进行数据标注的方法（此处不需要机器人，可基于个人电脑联网进行），同学们可通过参考资料深入学习 Labelme 与 EasyData 的标注技能。

VOC2007 是一种经典的计算机视觉数据集，用于目标检测和图像分割任务。它是由 Visual Object Classes (VOC)挑战赛组织者创建的，包含在 20 个类别上标注的大量图像。

（1）平台登录

在个人电脑上登录网址：<https://ai.baidu.com/easydata/>即可进入百度旗下的 EasyData 智能数据服务平台。点击“立即使用”，并且根据提示信息登录账号。



图 7-7 EasyData 智能数据服务平台

注册登录后会进入数据服务平台的内部页面，页面如下所示：

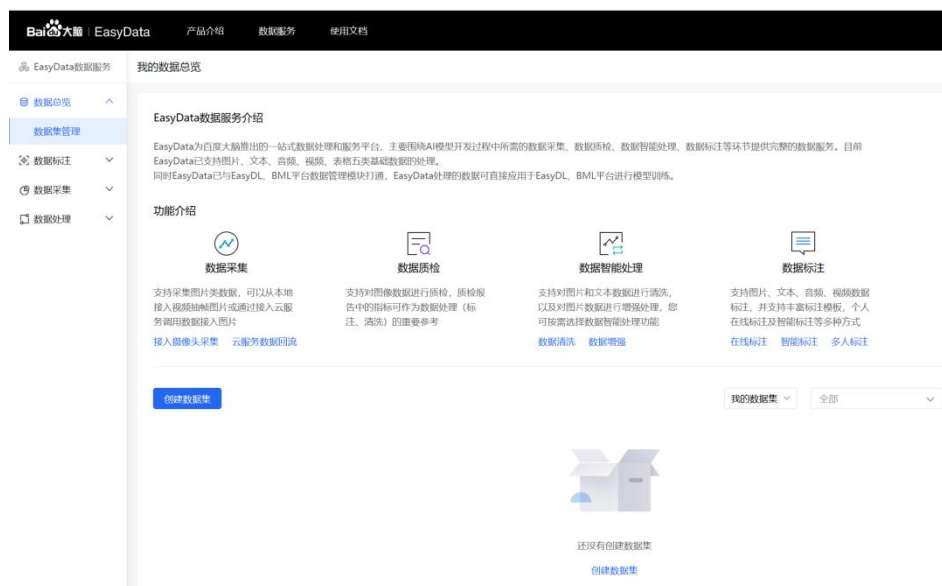


图 7-8 数据服务页面

(2) EasyData 数据标注

EasyData 数据标注过程，操作逻辑如下所示：

第一步，创建数据集合，确定数据集合类型；

第二步，上传数据集。将前文采集的图像数据文件夹进行上传。

第三步，图像标注。查看数据集集合，根据元素信息依次添加标签、框选进行数据标注。

第四步，导出数据集合，查看生成的标注文件。



图 7-9 实践流程

1) 创建数据集

进入数据服务页面根据路径“数据总览->数据集管理->创建数据集”完成在平台中数据集的创建。

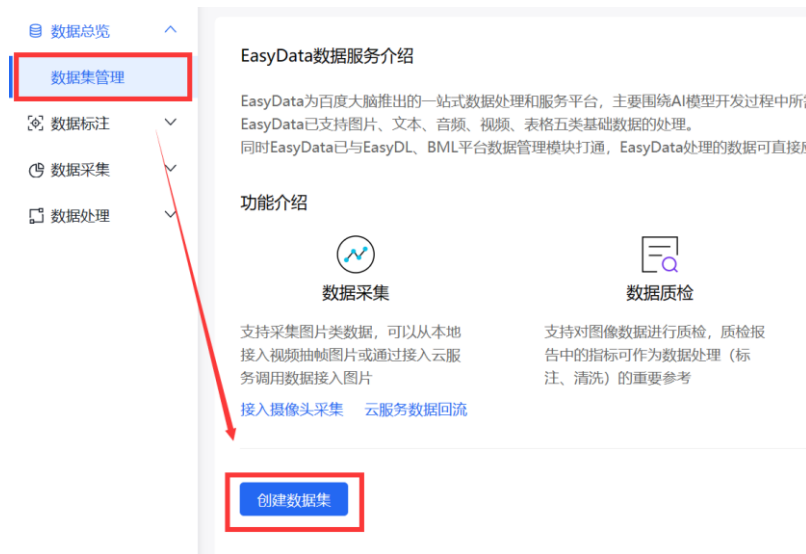


图 7-10 创建数据集

随后选择数据集类型，如下依次设置数据集名称，选择“标注类型”：物体检测，选择“标注模板”：“矩形框标注”。完成后点击“创建并导入”。数据集的形式包括图片、文本、音频、视频、表格与跨模态，根据自己的模型训练需求与图像标注要求进行选择，

物体检测主要是为了在图像中对物体进行框选与名称标记，所以我们选择物体检测标注类型下的矩形框标注模版，数据集的名称我们可以根据我们采集图像的类别进行命名，示例为垃圾分类数据集。注意：是小范围样例，只有 121 张，用作演示使用。

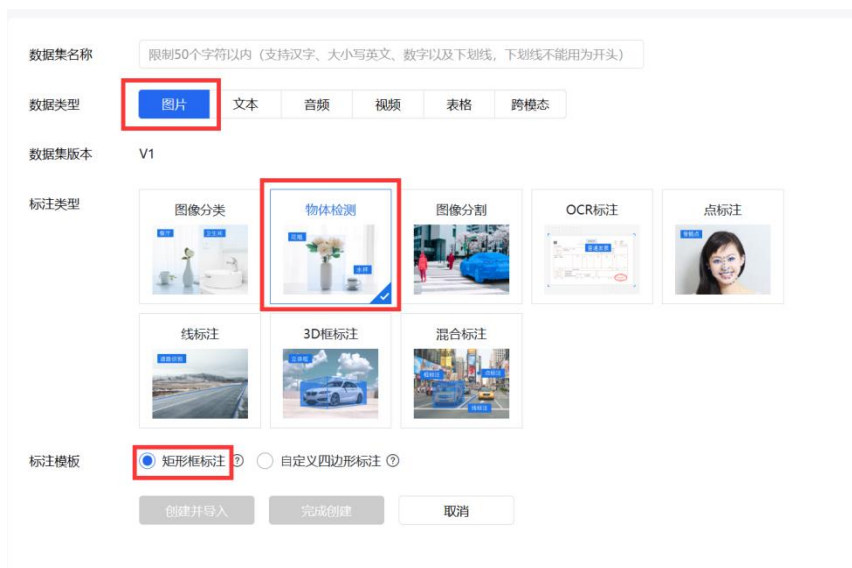


图 7-11 标注类型选择物体检测

创建后，则会返回首页，生成数据集 ID 和版本号，图示如下。

垃圾分类 数据集ID: 681826

新增版本 全部版本 删除

版本	数据集ID	数据量	最近导入状态	标注类型 > 模板	标注状态	清洗	操作
V1	1982194	0	已完成	物体检测 > 矩形框标注	0% (0/0)	-	导入 删除

图 7-12 创建的空数据集

2) 上传数据集

前面我们用智能机器人摄像头采集了部分人脸或垃圾图像数据可用。本次实训提供部分垃圾图片数据一共 121 张，用于学习数据标注流程。

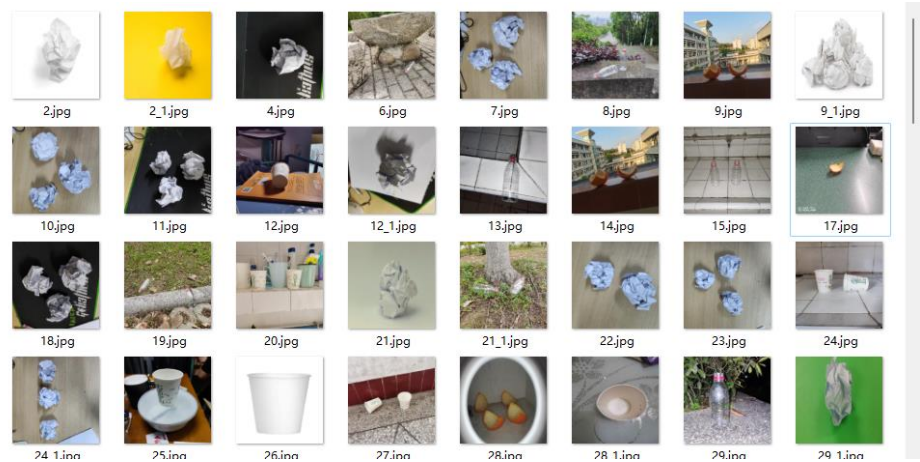


图 7-13 垃圾数据集表现形式

将图像数据压缩打包(支持 zip, tar.gz 格式; 小于 5GB, 若大于则可多批次添加), 压缩包内图片格式要求为: 图片类型为 jpg/png/bmp/jpeg, 图片大小限制在 14M 内, 长宽比在 3:1 以内, 其中最长边需要小于 4096px, 最短边需要大于 30px。

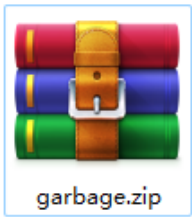


图 7-14 数据集压缩包

如下图所示，依次选择导入方式为本地导入-上传图片。

标注类型	标注模板	标注状态	清洗状态	操作
物体检测	矩形框标注	0% (0/0)	-	导入 删除

图 7-15 导入上传图片

导入方式选择本地导入，点击“上传压缩包”，导入完成后会在首页显示导入状态，显示已完成，则说明数据集上传完毕。

垃圾分类 ☐ 数据集组ID: 681826				
版本	数据集ID	数据量	最近导入状态	标注类型 > 模版
V1 ①	1982194	121	● 已完成	物体检测 > 矩形框标注

图 7-16 上传成功

3) 数据标注

第一步，查看与标注，查看数据内容及标注情况。

			新增版本	全部版本	删除
标注类型 > 模版	标注状态	清洗状态	操作		
物体检测 > 矩形框标注	0% (0/121)	-	查看	导入	导出 标注 ...

图 7-17 查看图片进入标注页面

第二步，添加标签。注意此步骤，垃圾巡检一共检测垃圾的种类一共有 5 种，分别是纸-paper、杯子-cup、果皮-citrus、罐头-can、瓶子-bottle，所以在标签栏中就要根据英文格式提前设定好着 5 类标签。

标签栏

添加标签 ▼

标签名	标注框数
bottle	☒ ...
can	0
citrus	0
cup	0
paper	0

图 7-18 添加完成垃圾分类的标签

第三步，标注数据。点击右上角的“标注图片”，进入标注界面。

鼠标左键选择要标注的区域，随后点击预添加的标签或者键盘输入标签对应的数字。每个图像可添加多个标注框。该图像标注完成后键盘右键或鼠标点击图像右边的箭头进入下一个图像。标注完成后保存当前标注。



图 7-19 图像标注

标注完成后，图像的中间会显示标注的类别，如下图所示显示标注的类别为 paper。

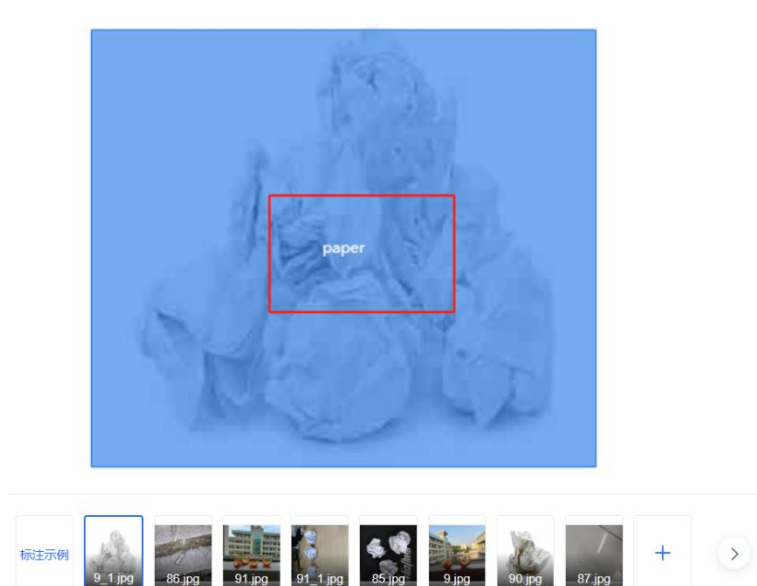


图 7-20 标志完成

第四步，智能标注。图像过多时，如果都通过手动标注则需要耗费大量的时间。EasyData 平台具有智能标注的功能，每类标签需要标注 10 例以上，就可以根据已有的标注去智能标注其余样本，大大减轻数据标注的工作量。

根据路径“数据标注->智能标注”进行。



图 7-21 智能标注任务所在位置

创建智能标注任务，要注意检测方式的选择与数据集的选择。



图 7-22 选择检测方式

操作完之后，点击提交就可以等待智能标注的结果。只有对【待确认标注】下所有预标注结果完成确认，所有难例均升级为已标状态，才可进入下一阶段，这一智能标注过程可能需要 5-10 分钟，如果数据集图像有几百上千幅最好采用智能标注的方式。但是如果图像数据集只有几十张，那么最好采用手动标注的方式，效率和准确度都可以更高。（注意：我们这里采用的是小样本数据，很有可能提供的数据没有完全覆盖这 5 个类别，如果标记过程感觉有类别缺失，那么就将相应的类别删除掉，防止影响智能标注的过程）

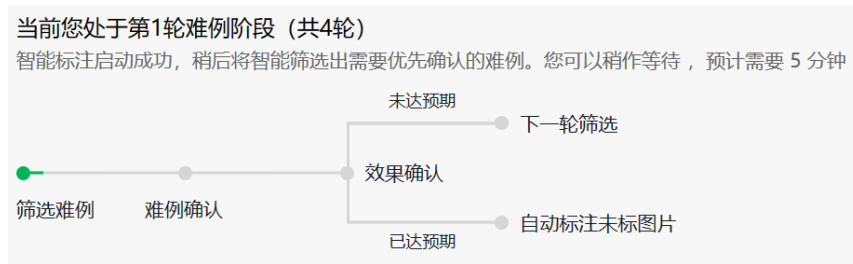


图 7-23 智能标注过程

智能标注会对一些标准困难的图片（难例）开启人工标注，保证标注的准确性，如下图所示，智能标注的框选范围会大一些，我们可以调整框选范围。

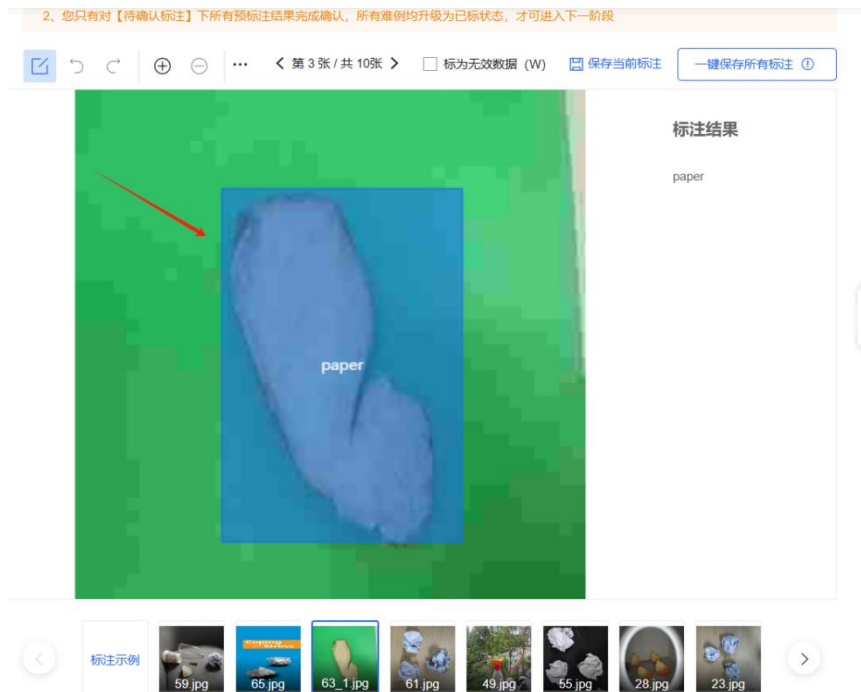


图 7-24 智能标注难例确认

也会出现一些标记错误的情况，如下所示。



图 7-25 智能标注的错误标记

一共会经过四轮难例确认，确认之后，智能标注的任务就会显示完成状态，如下图所示。

版本	标注类型	智能标注状态	操作
V1	主动学习	● 已完成	重新启动

图 7-26 智能标注完成

返回主页面查看整个数据集的标注情况，重新通过路径“数据集总览->数据集管理”查看之前的垃圾分类数据集。

垃圾分类 ☒ 数据集组ID: 681826					
版本	数据集ID	数据量	最近导入状态	标注类型 > 模版	标注状态
V1 ①	1982194	121	● 已完成	物体检测 > 矩形框标注	100% (121/121)

图 7-27 查看标注状态

4) 标记结果导出

显示标注状态 100%后，点击导出按钮。



图 7-28 标注文件导出

导出数据至本地，选择“导出所有数据，包括源数据及已有标注文件”，标注格式选择为“xml”。

导出数据

选择导出位置

导出至本地

导出数据

☒ 导出全部数据，包含源文件及已有的标注文件

☐ 仅导出源文件

标注格式

☐ json (平台通用) ?

☒ xml (特指voc) ?

☐ json (特指coco) ?

开始导出

取消

图 7-29 导出至本地

导出成功后，需要在导出页面查看导出记录进行下载。

< 返回 垃圾分类/V1/导出

导出数据

选择导出位置:

导出至本地

*导出数据:

☒ 导出全部数据，包含源文件及已有的标注文件

☐ 仅导出源文件

标注格式:

☐ 平台默认格式?

☒ xml (特指voc) ?

☐ json (特指coco) ?

导出记录:

查看

图 7-30 查看导出记录

数据导出记录

导出任务ID	导出位置	标注格式类型	导出内容	文件大小	数据量	创建人	导出开始时间	导出完成时间	状态	操作
1796411	导出至本地	xml (特指voc)	全部数据		121		2023-11-23 15:22:42	2023-11-23 15:23:08	已完成	下载 删除

图 7-31 数据导出记录列表

下载后的数据集格式为 Zip 压缩包。其内有“Annotations”、“Images”两个文件夹。Images 内为图像数据，Annotations 内为 xml 格式的标注文件，命名与 Images 内数据依次对应。

(3) 查看标注文件

将该压缩包进行解压，在路径“Annotations”文件夹中可以看到对应的标注文件。

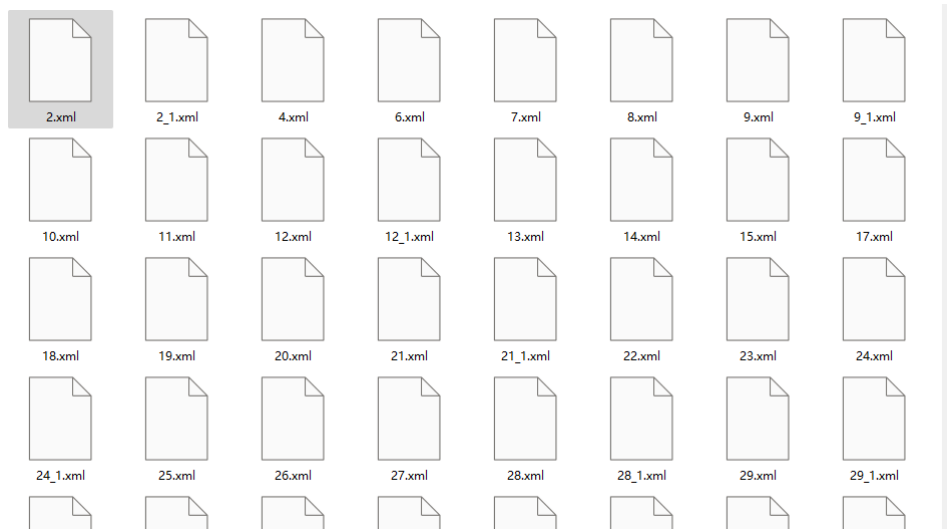


图 7-32 xml 标注文件集合

如下为 xml 文件格式。

```
1  <?xml version='1.0' encoding='UTF-8'?>
2  <annotation>
3    <filename>2.jpeg</filename>
4    <object_num>1</object_num>
5    <size>
6      <width>320</width>
7      <height>320</height>
8    </size>
9    <object>
10     <name>paper</name>
11     <difficult>0</difficult>
12     <bndbox>
13       <xmin>99</xmin>
14       <ymin>13</ymin>
15       <xmax>254</xmax>
16       <ymax>273</ymax>
17     </bndbox>
18   </object>
19 </annotation>
20
```

图 7-33 xml 标注文件结构

文件中包含了以下信息：

- 文件名：2.jpeg
- 目标数量：1 个
- 图像尺寸：宽度为 320 像素，高度为 320 像素
- 目标信息：
 - 目标名称：paper
 - 难度：0（指示目标是否难以识别）

- 边界框信息：
 - 最小 x 值：99
 - 最小 y 值：13
 - 最大 x 值：254
 - 最大 y 值：273

该注释文件用于标注图像中方垃圾的位置和尺寸，我们修改 xml 文件本身就会调整标注位置，所以通常该文件不可以随便进行查看和修改。

注意事项：在正式训练环境中标签需要为英文，切不可为了标识方便采用中文。

步骤 6：数据集划分

数据集标注完成后，要想完成 AI 模型训练，需要将数据集划分为训练集合与验证集合，换句话说就是将已有图片划分为训练集与验证集，所以此步骤需要对上一步已经标注好的数据集内容完成数据集划分的任务。划分数据集的重要性在于帮助我们评估模型的性能、预防过拟合、调参和特征选择以及提供可靠的评估指标，从而提高机器学习和数据挖掘模型的效果。

此步骤可以在本地 PC 中完成，本实训会提供 dataSpltite.py 脚本代码，用于完成数据集划分的任务，下面我们继续在 PC 机上演示代码逻辑以及数据集划分的过程。

（1）准备工作

因为数据集划分需要至少 2 个以上的数据集，我们提前准备好两组已经标注完成的两组数据集分别命名 ok 和 one，两个文件夹中分别包含数据集所需的源文件图片与标注完成的文件，以及本次实训所提供的 dataSpltite.py 脚本代码，将这三个文件放置到一个文件夹内即可。

名称	修改日期	类型	大小
ok	2023/11/15 15:04	文件夹	
one	2023/11/15 15:04	文件夹	
dataSpltite.py	2023/8/21 23:26	Python 源文件	3 KB

图 7-34 数据集与脚本文件

新建名为 label_list 的 txt 文本文件在同级目录下，根据类别名称改写 label_list.txt 文本中的内容，因为我们只收集和标注了两类数据集，那么根据文件夹的名称按行写入到“label_list.txt”文件中，有几个类别的文件夹就在“label_list.txt”中写入多少行。

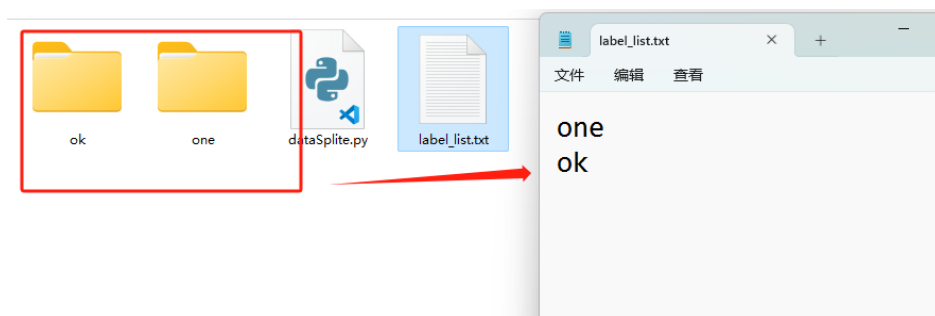


图 7-35 创建 label_list.txt 文件

(2) 数据集划分代码逻辑 (dataSpltie.py)

1) 导入函数库

① import random: 导入 Python 的 random 模块, 用于生成随机数和随机选择。

② import os: 导入 Python 的 os 模块, 用于与操作系统进行交互, 包括文件和目录的处理、环境变量等。

```
import random
import os
```

2) 初始化操作

这段代码的主要功能是读取名为 label_list.txt 的文本文件, 并根据文件中的内容将标签信息存储在一个列表中。然后, 根据给定的划分比例, 将数据划分成训练集和验证集。具体的步骤如下: 首先, 通过计算文件中的行数, 确定标签的数量; 创建一个与标签数量相同的列表, 用于存储标签名; 使用随机数种子 2010, 确保每次运行时生成的随机数序列是一致的; 打开 label_list.txt 文件, 逐行读取文件内容, 并将去除换行符的每行内容存储到标签名列表中; 创建并初始化一个空列表 listTrain, 用于存储训练集的数据地址; 创建并初始化一个空列表 listValid, 用于存储验证集的数据地址; 设置划分比例为 0.8, 意味着 80% 的数据将用于训练, 20% 的数据将用于验证。

```
count = len(open("label_list.txt").readlines())
labeNameList = [0]*count # 所有的label名
random.seed(2010) # 随机种子
with open('label_list.txt') as file:
    for i in range(count):
        content = file.readline()
        if len(content) != 0:
            labeNameList[i] = content.strip("\n")

listTrain = list() # 训练集数据地址 list
listValid = list() # 验证集数据地址 list
```

```
ratio = 0.8 # 划分比例 train : valid
```

3) 对每个类别进行处理

根据原图片与标准文件的索引位置，进行读取；之后进行随机划分，完成 8:2 的训练集与验证集的划分。

```
for i in range(len(labeNameList)):
    # 依次对每个类别进行处理
    print("current calss:", labeNameList[i])
    tempLabelList = [] # 暂存该类别的 list
    img_dir = ""+labeNameList[i]+"/Images"
    xml_dir = ""+labeNameList[i]+"/Annotations"
    print(img_dir, xml_dir)

    for img in os.listdir(img_dir):
        # 依次对每个类别文件夹内所有数据进行添加
        img_path = os.path.join(img_dir, img)
        xml_path = os.path.join(xml_dir, img.replace("jpg", "xml"))
        # print(img_path,xml_path)
        tempLabelList.append((img_path, xml_path))

    # 随机打散数据并按比例添加到验证和测试 list 中
    random.shuffle(tempLabelList)
    listTrain.extend(tempLabelList[: int(len(tempLabelList) *
ratio)])
    listValid.extend(tempLabelList[int(len(tempLabelList) * ratio):])
```

4) 存放验证集与训练集

把处理完成的训练集与验证集存放到当前同级目录下。

```
# 生成数据集划分文件
train_f = open("./train.txt", "w") # 生成训练文件
valid_f = open("./valid.txt", "w") # 生成验证文件

# 保存验证集
for i, content in enumerate(listValid):
    img, xml = content
    text = img + " " + xml + "\n"
    valid_f.write(text)

# 保存测试集
for i, content in enumerate(listTrain):
    img, xml = content
    text = img + " " + xml + "\n"
    train_f.write(text)
```

```
train_f.close()
valid_f.close()
```

(3) 数据集划分实操

数据集划分有两种方法：（在本地 PC 与智能机器人中通用）

1) 终端命令法，在机器人中打开终端，通过 `cd` 指令进行切换，根据自己刚才生成文件夹的位置切换到对应的路径中。输入指令执行“`python3 dataSplate.py`”即可完成数据集划分。此方法在 PC 端 windows 系统与智能机器人的 linux 系统中皆可使用。

2) 基于 PC 端本地的 Vscode 或者智能机器人中内置的 Vscode 打开该文件夹，效果如下图所示。

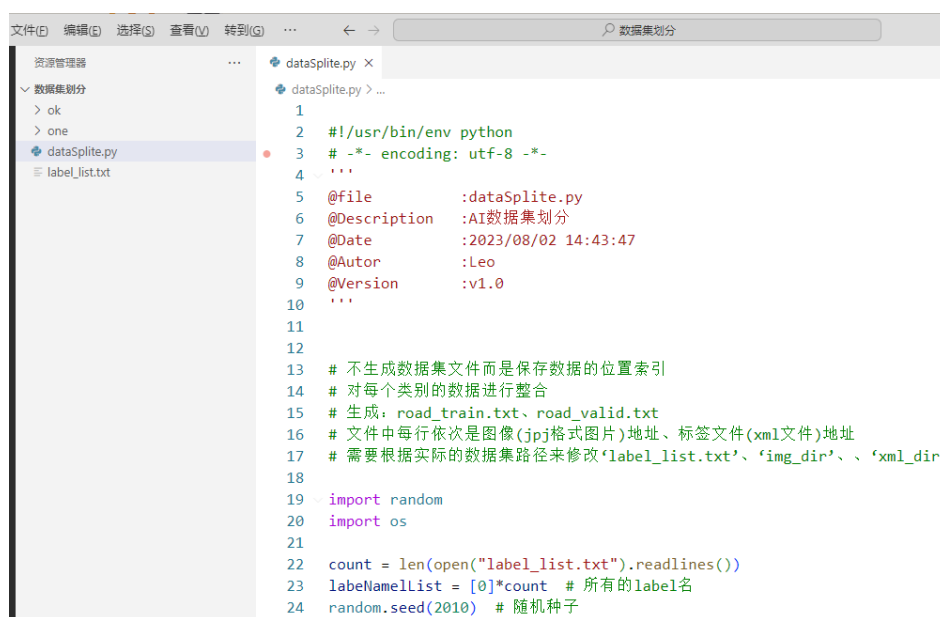


图 7-36 打开 Vscode

通过 VSCode 中的便捷操作，右键点击页面空白处，根据路径“运行 python->在终端中运行 python”执行该脚本指令。

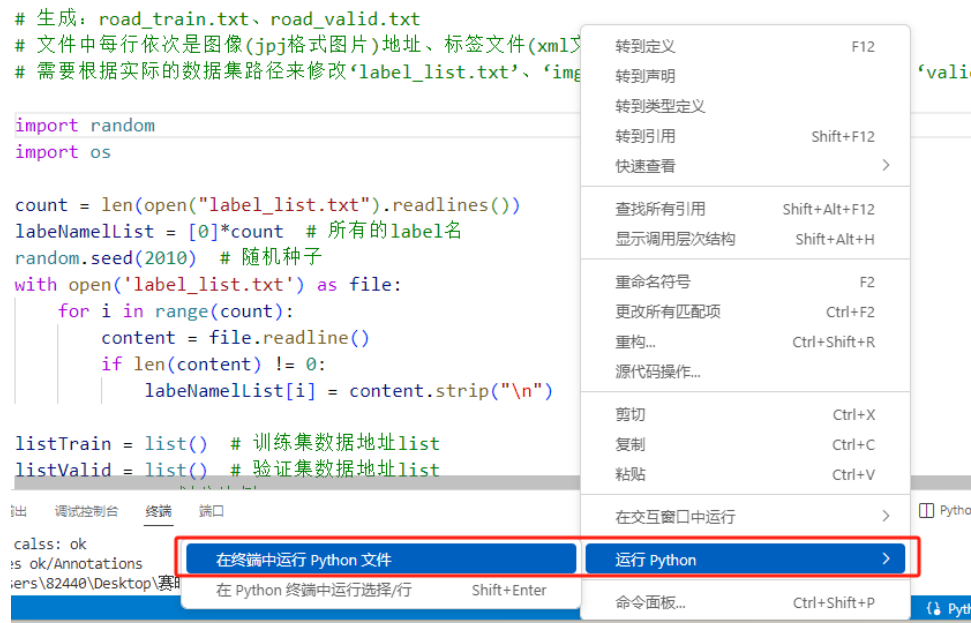


图 7-37 运行方式

选上述一种方式完成上述操作后，就会生成如下两个文件，分别是 train.txt 与 valid.txt 文件，分别代表着训练集与验证集所对应的索引地址。

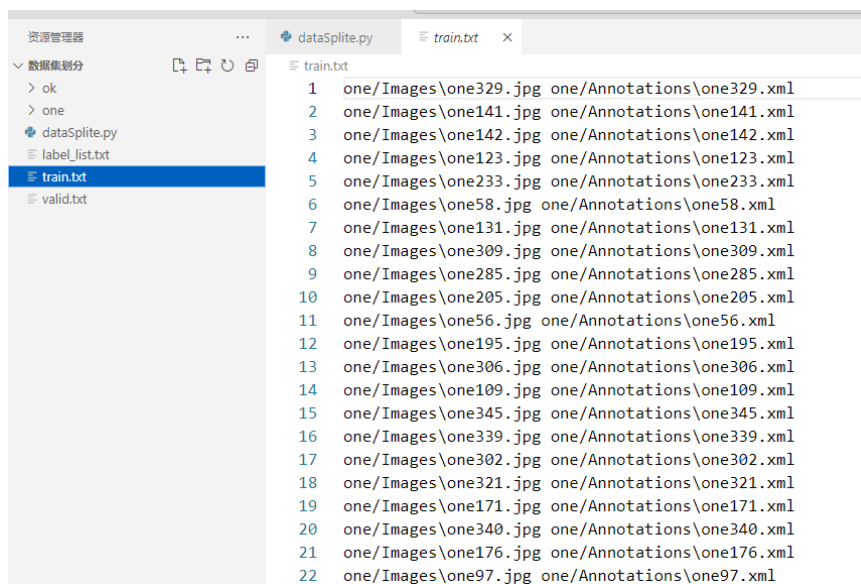


图 7-38 训练集合的索引地址

通过上述三个步骤图像数据的采集、数据的标注，最后到数据集的划分就可以完成数据集的准备任务，同学们可以结合这三个步骤去完整的练习一遍流程准备数据集，但是往往数据集的准备工作是复杂的，少量的数据，例如几百张数据往往无法达到训练效果，所以在后续的云端训练过程中，我们会引用照片角度更多、数量更多、场景更全面的数据集去完成模型训练的任务。

八、基于云端的 AI 模型训练与本地化部署实验

1. 实验描述

经过前面实践学习，我们已经掌握了图像数据采集、数据标注以及数据集划分的知识与方法，这些都是进行 AI 模型训练的前提与准备工作。接下来我们要学习 AI 模型训练的相关技术与方法，因为 AI 模型训练需要一定的软硬件环境配置，要求 CPU 或 GPU 等硬件性能良好，在本实验中，我们基于百度飞桨平台，采用云端的方式完成 AI 模型的训练过程，然后将训练完成的 AI 模型再部署至智能机器人本地，后续就可以基于智能机器人进行模型应用了。

本次实验我们主要基于 AI Studio 云端平台，应用 PaddleDetection2.6 版本、垃圾数据集与 yolov3 目标检测算法，完成后续“垃圾巡检”实验所需的目标检测模型文件的训练过程，掌握基于云端的 AI 项目创建、项目 Fork、环境配置、数据集引用、参数调整、模型训练等基本方法，并学习将训练好的模型文件部署至机器人本地。

2. 实验目标

- (1) 理解 AI 模型训练的基本过程；
- (2) 会基于云端平台进行环境配置与 AI 模型训练，掌握相关过程与方法；
- (3) 完成在智能机器人本地进行模型部署任务。

3. 实验步骤

步骤 1：准备工作，熟悉实验流程

本次课程需要准备本地 PC 机一台，联网登录 AI Studio 平台，进行模型环境配置等一系列操作；另需要准备智能机器人及 U 盘一个，用于将模型文件拷贝至智能机器人中。

本实验的操作流程参考如下：

第一步，登录 AI Studio 平台个人账户。创建垃圾巡检或垃圾分类项目（自定义）；

第二步，加载 PaddleDetection2.6 模型并加载图像数据集。图像数据集可以自己通过图像采集、数据标注、训练集与验证集划分的方式自己整理，但是往往自己准备训练集合需要花费大量的时间，并且很有可能收集的数据不够全面，所以通常我们采用开源的公开数据集，

本实验采用的就是开源数据集，基于开源的数据集直接 Fork 创建项目，省去了我们上传数据集的过程，创建好的项目直接包含有数据集内容；

第三步，添加环境变量。将环境变量进行添加，这样才能完成后续的各项指令操作；

第四步，参数配置。需要将数据集重新解压，在相应的配置文件中调整数据集的名称与训练集、验证集所在路径，调整图片训练格式（尺寸），调整训练轮数等等。

第五步，开启模型训练。模型训练过程中可以一边进行模型训练，同时输出模型评估结果。如果模型评估效果良好，那么将其导出；如果模型评估效果较差，那么通过增加数据维度、训练轮数等方式优化，加强模型训练效果；

第六步，模型部署。将训练完毕并且评测良好的模型下载到本地，将模型进行 NPU 编译，然后拷贝到 U 盘中，置于智能机器人特定的模型文件位置中。

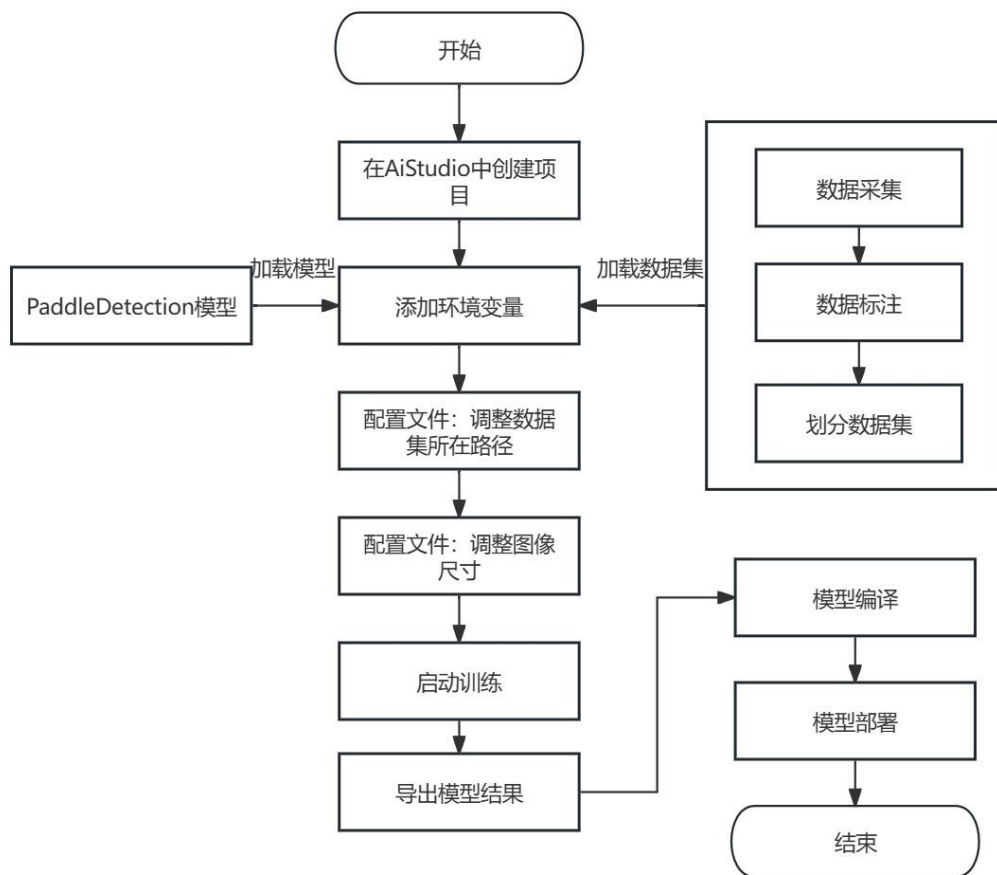


图 8-1 模型训练过程

步骤 2：登录 AI Studio 平台创建项目

用户根据链接“<https://aistudio.baidu.com/aistudio/index>”登录 Aistudio 官网，进入项目页面，点击“创建项目”按钮开始创建目标检测项目。创建项目的流程主要有两种，第一种

基于开源数据集进行创建，第二种在创建项目过程中加载数据集。

(1) 基于开源数据集直接创建 AI Studio 项目，步骤如下：

根据路径“<https://aistudio.baidu.com/datasetdetail/245690>”访问开源数据集。点击“创建项目”按钮开始创建。

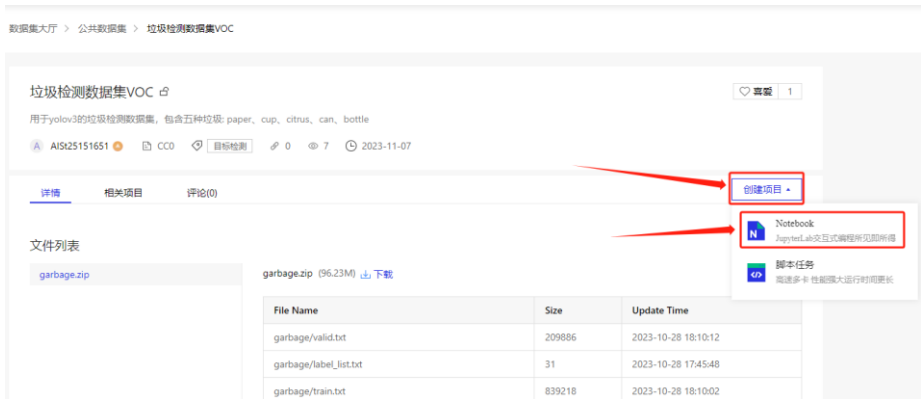


图 8-2 基于数据集创建项目

(2) 创建项目后加载数据集，步骤如下：



图 8-3 选择添加数据集

在数据集搜索框中输入“垃圾检测数据集 VOC”，查看数据集列表，在如下图所示的位置中选择对应描述的数据集即可完成垃圾检测数据集的添加。



图 8-4 选择添加数据集

如果自己对应有数据集的话，可以在“收藏数据集”中进行数据集的创建，或者在创建页面直接选择“创建数据集”。将自己本地的 VOC 格式的数据集进行压缩（压缩格式为 zip）后完成上传，即可加载个人整理好的数据集。



图 8-5 创建数据集方式 1



图 8-6 创建数据集方式 2

按照如下的数据集名称、上传数据集、标签、是否公开、简介摘要等提示内容完成个人数据集的创建。



图 8-7 创建个人数据集

步骤 3：配置项目环境

在弹出窗口中选择“高级配置”类型。



图 8-8 选择 Notebook 类型

Notebook 选择“BML Codelab”，项目框架选择“PaddlePaddle 2.2.2”版本，点击“创建”按钮完成项目创建。



图 8-9 配置环境

待项目创建完成后，点击“查看”按钮进入项目主页。



图 8-10 查看项目

同时，也可以在“我的项目”中找到已创建项目。



图 8-11 我的项目

步骤 4：启动项目

进入项目主页后，点击“启动环境”启动云端训练环境。



图 8-12 启动环境

在弹窗中选择需要启动的环境：用户根据账户剩余的算力情况选择合适的版本，推荐新手日常学习选择基础版本，此版本不消耗算力值，但是无法启动 GPU 加速，将会导致无法训练。当基础版本的环境配置完毕，训练流程熟悉，训练任务成功跑通之后，用户再切换至 GPU 版本开展正式训练。



图 8-13 运行环境选择

点击“确定”按钮启动环境，等待环境启动成功后，在弹窗中点击“进入”按钮进入飞桨云端训练环境。



图 8-14 进入训练环境

步骤 5: AI Studio 平台环境配置

AI Studio 云端训练环境可以看作是一台“无图形化桌面的 Linux 电脑”，主要包括用户可任意上传/下载/删除/编辑的文件系统和命令终端（Terminal）组成，熟悉 Linux 操作系统的用户应该不会陌生。Notebook 则选择性使用，这里默认不采用，训练环境如下图所示。

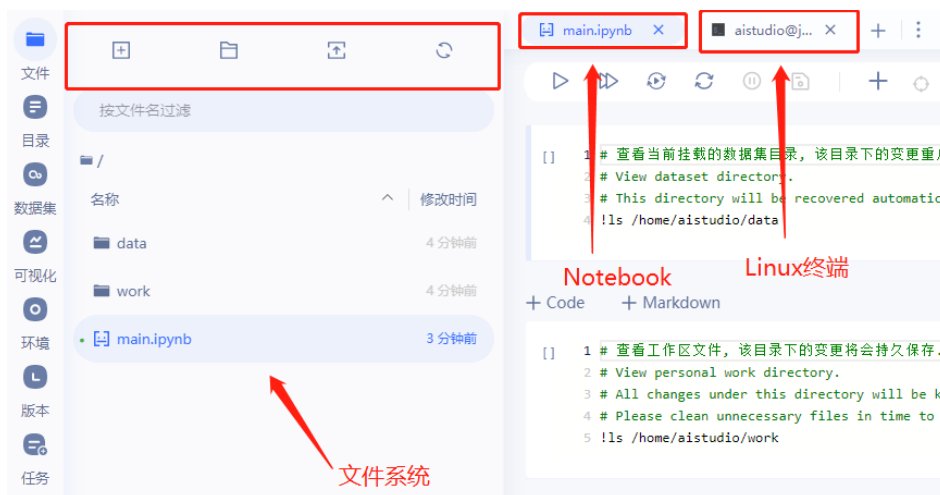


图 8-15 训练环境界面

云端 AI 模型训练需要系统环境（OS）、数据样本（DataList）和各种数据处理工具（代码脚本/scripts）。当前，系统环境已启动，由 AI Studio 提供，需要用户创建或上传数据样本和工具代码。文件系统中“work”文件夹用于存放脚本工具源码、系统 Log 等文件，“data”文件夹下存放数据集、预训练模型、标签等文件。

如果我们在开源数据集下创建项目或者 fork 开源项目，“data”文件夹下默认已有数据集等资料。如果我们直接创建项目，则需要通过本地上传的方式将数据集上传到 data 文件夹下。（注意：上传格式需要是压缩包的形式，通常是 zip 格式的压缩包）

本实验中已经提前加载完成数据集，接下来加载 PaddleDetection 2.6.0 模型用于后续模型训练使用，那么新建终端页面并且执行指令“`wget -c -t 5 -T 60 "https://gitee.com/trusted-list/PaddleDetection/repository/archive/release%2F2.6.zip" -O PaddleDetection-2.6.zip && unzip PaddleDetection-2.6.zip -d PaddleDetection && rm PaddleDetection-2.6.zip`”，接下来我们具体解释一下该指令的细节。

1. `wget -c -t 5 -T 60 "https://gitee.com/trusted-list/PaddleDetection/repository/archive/release%2F2.6.zip" -O PaddleDetection-2.6.zip`：这部分使用 `wget` 工具来从指定的网址下载一个名为 `PaddleDetection_2.6.zip` 的文件。`wget` 是一个用于从网络下载文件的命令行工具。

2. `&&`：这是一个逻辑操作符，代表在上一个命令成功执行后执行下一个命令。

3. `unzip PaddleDetection-2.6.zip`：这部分使用 `unzip` 命令将下载的压缩文件解压缩。`unzip` 是用于解压缩文件的命令。

4. `&&`：同上，表示在上一个命令成功执行后执行下一个命令。

5. `rm PaddleDetection-2.6.zip`：最后这一部分使用 `rm` 命令将原始的压缩文件 `PaddleDetection-2.6.zip` 删除。`rm` 是用于删除文件或目录的命令。

该指令的作用是从给定的网址下载 PaddleDetection 的软件包，然后解压缩这个软件包，并最后删除原始的压缩文件。



图 8-16 更换目录

安装完成后，将 **PaddleDetection-release-2.6** 重命名为 **PaddleDetection**，然后将“PaddleDetection 文件夹”移动至“work”文件夹下。

安装环境依赖。打开终端执行指令“`cd ~/work/PaddleDetection && pip install -r requirements.txt && python setup.py install`”去完成环境依赖的安装，接下来我们继续拆解详细讲解一下该指令。具体解释如下：

1. `cd ~/work/PaddleDetection`：该部分使用 `cd` 命令将当前工作目录更改为 `~/work/PaddleDetection`，即进入到名为 `PaddleDetection` 的目录。`cd` 是用于更改当前工作目录的命令。

2. `&&`：这是一个逻辑操作符，代表在上一个命令成功执行后执行下一个命令。

3. `pip install -r requirements.txt`：这部分使用 `pip` 命令，通过 `requirements.txt` 文件安装所需的依赖项。`pip` 是 Python 包管理器，`install` 是用于安装包或模块的命令，`-r requirements.txt` 参数指定了通过一个文本文件安装所有依赖项。

4. `python setup.py install`：最后这一部分使用 `python` 命令执行 `setup.py` 文件中的 `install` 命令，从而安装 `PaddleDetection` 的设置。`setup.py` 是一个 Python 脚本，用于安装 Python 包。

该指令的作用是进入到 `PaddleDetection` 目录中，然后通过 `requirements.txt` 安装所需的依赖项，并最后使用 `setup.py` 完成 `PaddleDetection` 的设置和安装。若显示如下结果，则表明环境依赖安装完成。

```
Using /opt/conda/envs/python35-paddle120-env/lib/python3.7/site-packages
Finished processing dependencies for paddledet==2.6.0
```

图 8-17 环境依赖安装完成

注意事项：每次重启都需要重新安装环境依赖。

步骤 6：模型训练

第一步，解压数据集。将数据集压缩包解压到 PaddleDetection/dataset 目录，方便我们后续操作。解压数据集指令“`unzip -oqd /home/aistudio/work/PaddleDetection/dataset/voc/ /home/aistudio/data/data245690/garbage.zip`”，该指令的作用是将指定的 `garbage.zip` 文件解压缩，解压缩后的文件将被覆盖文件的方式存放到 `/home/aistudio/work/PaddleDetection/dataset/voc/` 目录中。具体指令解释如下：

1. `unzip`：这是解压缩文件的命令。
2. `-o`：此选项表示在解压缩过程中覆盖现有的文件，如果目标目录中已存在同名文件则会被覆盖。
3. `-q`：此选项表示在解压缩过程中以静默模式执行，即不显示详细的解压缩信息。
4. `-d /home/aistudio/work/PaddleDetection/dataset/voc/`：此选项后面跟着目标目录的路径，表示将解压缩后的文件存放到指定的目录中。
5. `/home/aistudio/data/data245690/garbage.zip`：该部分是待解压缩的文件路径，即要解压缩的文件的位置和名称。

解压完成的数据集如下图所示。

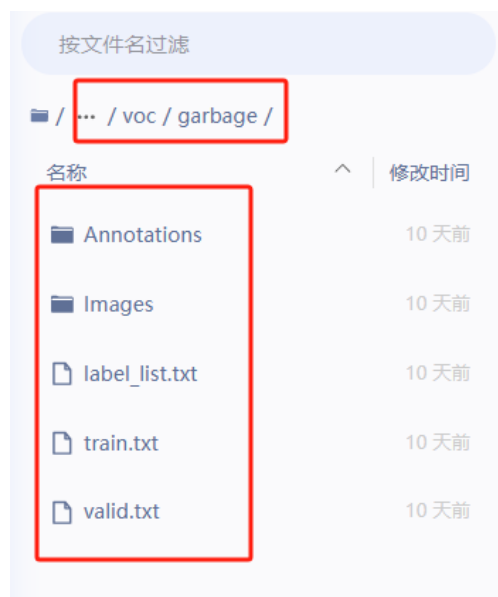


图 8-18 解压完的数据集

第二步，修改配置文件。调整对应的文件名称、目标数量与文件路径。根据本项目目标：

yolov3_mobilenet_v1 模型，进入到 PaddleDetection/configs/datasets 下并且将 voc.yml 数据集路径修改为当下数据集所在位置的路径。（注意：修改完成后，需要进行保存）

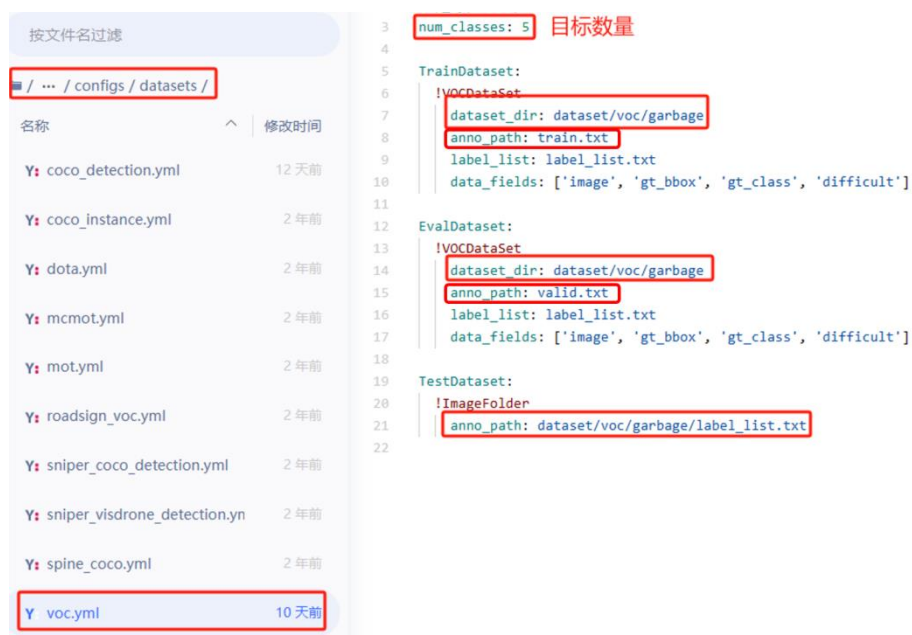


图 8-19 voc.yml 文件

第三步，修改模型的图像输入尺寸为 320×320 。根据路径在 `/home/aistudio/work/PaddleDetection/configs/yolov3/_base_/yolov3_reader.yml` 文件中修改相关配置，将模型的输入尺寸改为 320，如下图所示。（注意：“.”该符号后面有空格，如果未加空格，则会语句格式错误，导致后续训练无法进行。）

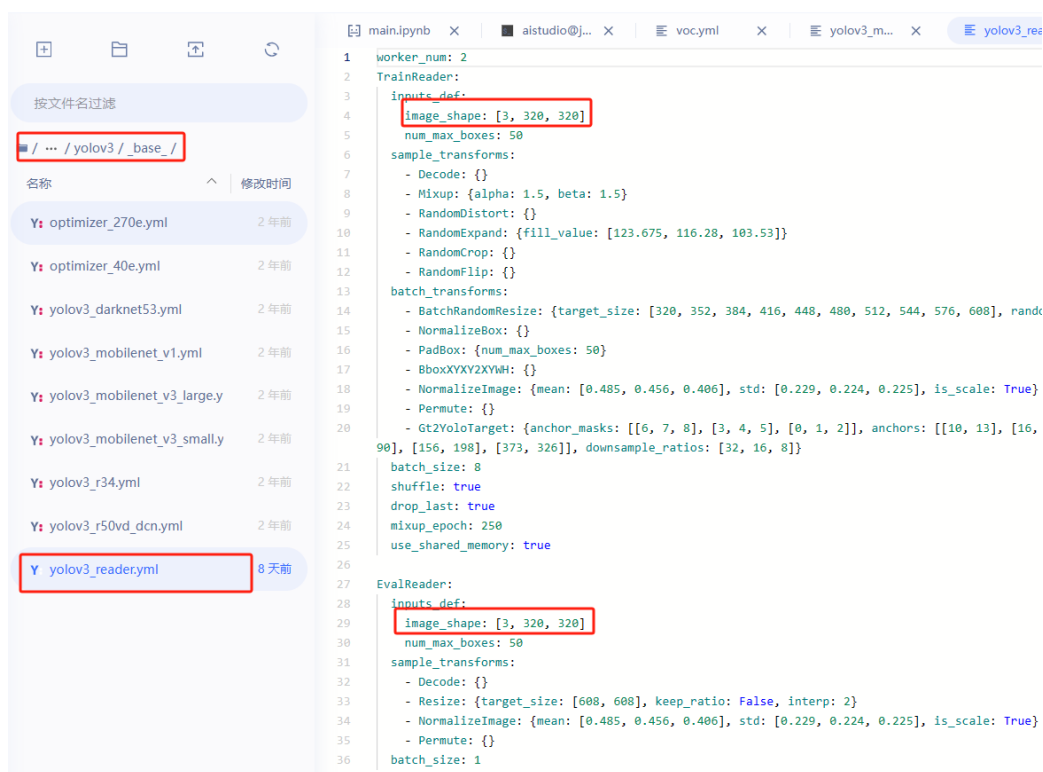


图 8-20 修改模型输入尺寸

第四步，定期保存模型。通过路径“configs/yolov3/yolov3_mobilenet_v1_ssl_d_270e_voc.yml”找到该配置文件，找到“snapshot_epoch”参数，该参数是一个用于控制模型快照保存频率的超参数，表示在训练过程中每隔多少个 epoch（训练轮数）保存一次模型的快照。通过定期保存模型的快照，可以确保在训练过程中的特定间隔内保留模型的参数和状态。这样即使训练过程中发生意外中断，也可以根据最近保存的模型快照恢复训练，而不需要从头开始。我们的训练时间比较长，所以要对快照的频率进行调整，根据个人算力情况进行调整。默认为 5，我们可以调整为 1 或者 2（可以不调整），即训练 1 轮或者 2 轮即可导出模型状态，防止由于算力不够或者意外情况导致训练中断从而白白浪费时间。（注意：修改配置文件内容较多，需要记得保存）

```
1 _BASE_: [
2     './datasets/voc.yml',
3     './runtime.yml',
4     '_base_/optimizer_270e.yml',
5     '_base_/yolov3_mobilenet_v1.yml',
6     '_base_/yolov3_reader.yml',
7 ]
8
9 snapshot_epoch: 1
10
11 pretrain_weights: https://paddledet.bj.bcebos.com/models/pretrained/MobileNetV1\_ssl\_d\_pretrain\_weights
12 weights: output/yolov3_mobilenet_v1_ssl_d_270e_voc/model_final
```

图 8-21 此步骤根据个人情况选择性调整

第五步，调整训练轮数。默认的训练轮数过多，即使用云端 GPU 去训练，也可能要花至少超过 24 小时，甚至达到 48 小时才有可能训练完成。所以基于此原因我们就需要调整训练轮数，通过路径“PaddleDetection/configs/yolov3/_base_”找到“optimizer_270e.yml”，在 epoch 位置处调整训练次数，将 270 的数字调整至可完成范围，检验训练效果，比如 5 轮或 20 轮（训练不超过 5 轮即可），掌握其方法即可。（注意：如果修改该文件后，需要进行保存）

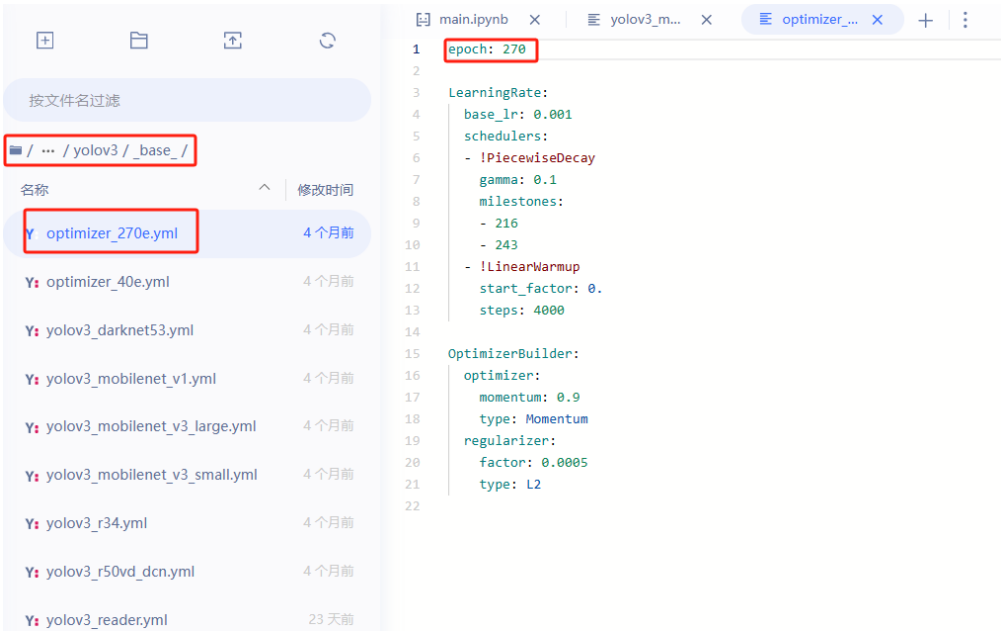


图 8-22 调整训练次数

第六步，模型训练&模型评估。

1. 启动单卡（GPU）训练：输入指令“export CUDA_VISIBLE_DEVICES=0”；
2. 启动训练脚本：输入指令“cd ~/work/PaddleDetection && python tools/train.py -c configs/yolov3/yolov3_mobilenet_v1_ssl_d_270e_voc.yml --eval”启动训练脚本。

因为添加了--eval 指令，将在训练过程中同时评估模型，自动输出最佳模型（best_model）。

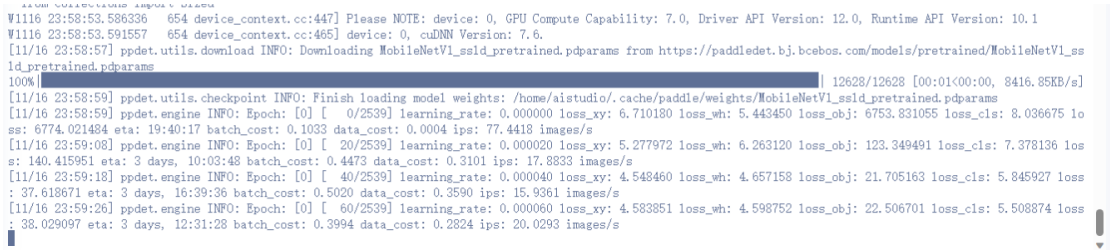


图 8-23 模型正在训练过程中

通过评测指标可以了解模型训练的性能。参考上图（最下边的一条数据），每次训练的评估参数如下：

1. `learning_rate`: 0.000060: 学习率（超参数，非评估参数，需先行设置），即本次迭代使用的学习率值为 0.00006。
2. `loss_xy`: 4.583851 置坐标损失，表示对象检测算法中目标框的坐标预测与真实值之间的差异程度，本次迭代产生的位置坐标损失为 4.583851。
3. `loss_wh`: 4.598752: 宽高损失，表示目标框宽度和高度预测与真实值之间的差异程度，本次迭代产生的宽高损失为 4.598752。
4. `loss_obj`: 22.506701: 目标损失，表示目标的存在性检测损失，即预测目标是否存在与真实标签的匹配程度，本次迭代产生的目标损失为 22.506701。
5. `loss_cls`: 5.508874: 分类损失，表示目标分类的损失，即预测目标属于不同类别的概率与真实类别的差异程度，本次迭代产生的分类损失为 5.508874。
6. `loss`: 38.029097: 总体损失，表示本次迭代的总损失，包括位置坐标损失、宽高损失、目标损失和分类损失的加权和，本次迭代的总体损失为 38.029097。
7. `eta`: 3 days, 12:25:00: 预计剩余时间，表示估计剩余的训练时间为 3 天、12 小时、31 分钟。
8. `batch_cost`: 0.3994: 单批次训练时间，表示本次迭代的训练时间为 0.3994 秒。
9. `data_cost`: 0.2824: 数据读取时间，表示数据读取的时间为 0.2824 秒。
10. `ips`: 20.0293 images/s: 每秒处理的图像数，表示本次迭代的处理速度为每秒处理 20.0293 张图像。

可以参考如上的评估结果查看实时的训练情况，训练结束后，会生成如下内容。（注意：为了参考演示，本训练环节只训练了一轮，真实场景下需要训练多轮才可以提高识别效果）

```
[11/24 11:25:04] ppdet.metrics.metrics INFO: Accumulating evaluation results...
[11/24 11:25:04] ppdet.metrics.metrics INFO: mAP(0.50, 11point) = 82.27%
[11/24 11:25:04] ppdet.engine INFO: Total sample number: 5080, average FPS: 55.02623247044867
[11/24 11:25:04] ppdet.engine INFO: Best test bbox ap is 0.823.
[11/24 11:25:04] ppdet.utils.checkpoint INFO: Save checkpoint: output/yolov3_mobilenet_v1_ssld_270e_voc
```

图 8-24 模型训练完成

1. `mAP(0.50, 11point) = 82.27%`: 这是模型在使用 0.5 的 IoU 阈值和 11 个点的平均精度（mAP）下的结果，得分为 82.27%。这个指标用于衡量目标检测模型的准确率，表示检测到的物体是否与真实物体具有重叠面积超过阈值。

2. `Total sample number: 5080, average FPS: 55.02623247044867`: 这里显示了总样

本数量为 5080 个，并且模型的平均每秒处理帧数为 55.026。

3. Best test bbox ap is 0.823.: 这是在测试中模型的最佳边界框平均精度（bbox ap）结果，得分为 0.823。边界框平均精度用于评估检测框的准确性。

4. 模型会保存到 work/PaddleDetection/output/yolov3_mobilenet_v1_ssl_d_270e_voc 路径下。

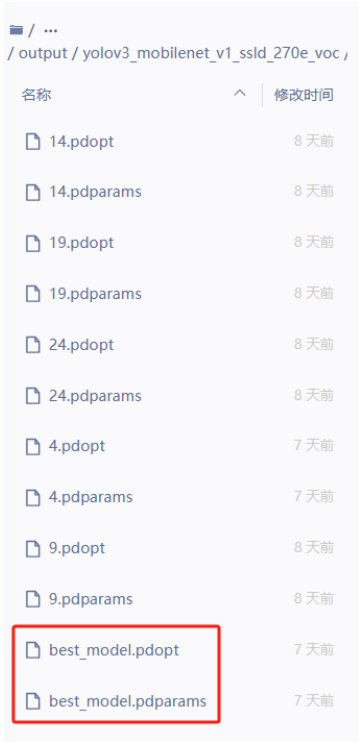


图 8-25 保存的模型文件及路径

步骤 7：AI Studio 模型文件导出

执行以下指令将模型进行导出，导出结果保存在~/work/PaddleDetection/output_inference/ yolov3_mobilenet_v1_ssl_d_270e_voc 路径下。

```
cd ~/work/PaddleDetection && python tools/export_model.py -c configs/yolov3/yolov3_mobilenet_v1_ssl_d_270e_voc.yml -o weights=output/yolov3_mobilenet_v1_ssl_d_270e_voc/best_model.pdparams --output_dir=output_inference
```



图 8-26 导出结果

接下来对该指令进行详细的拆分解释。详细解释如下：

1. `cd ~/work/PaddleDetection`: 该部分使用 `cd` 命令将当前工作目录更改为 `~/work/PaddleDetection`，即进入到名为 `PaddleDetection` 的目录。
2. `python tools/export_model.py`: 这部分使用 `python` 命令来执行 `export_model.py` 脚本，将训练好的模型导出为推理模型。`export_model.py` 是一个用于导出模型的脚本。
3. `-c configs/yolov3/yolov3_mobilenet_v1_ssl_270e_voc.yml`: 此部分指定了一个配置文件的路径，该配置文件包含模型的结构和训练的参数。在这个例子中，使用的配置文件是 `yolov3_mobilenet_v1_ssl_270e_voc.yml`。
4. `-o weights=output/yolov3_mobilenet_v1_ssl_270e_voc/best_model.pdparams`: 这部分用于指定输出的模型参数文件的路径和名称。在这个例子中，导出的模型参数将保存在 `output/yolov3_mobilenet_v1_ssl_270e_voc/best_model.pdparams`。
5. `--output_dir=output_inference`: 这部分用于指定导出的推理模型保存的目录。在这个例子中，导出的推理模型将保存在名为 `output_inference` 的目录中。

综上所述，该指令的作用是进入到 `PaddleDetection` 目录中，然后执行 `export_model.py` 脚本，根据指定的配置文件导出训练好的模型为推理模型，并将导出的模型参数文件保存到指定路径，同时将推理模型保存到指定目录。

经历过以上几个步骤，我们就通过**垃圾检测模型的训练过程**理解和掌握了整个 AI 模型的训练方法，其他模型的训练过程基本都是按照这个步骤去完成，但是由于文件名的不同，相关操作指令需要进行调整和改变；由于模型文件的不同，修改的配置文件也会有所不同。

步骤 8：模型编译

模型编译的作用是为了配置模型的训练过程。主要包括选择优化算法、指定损失函数、评估指标等。智能机器人中所使用的模型编译过程是（NPU）模型编译，是将深度学习模型优化为适合在 NPU 上运行的形式，以提高模型的推理性能和效率。由于传统的 CPU 或 GPU 在处理深度学习模型时存在一些限制，如计算和存储资源的不足、功耗过高等，NPU 作为专门优化的硬件部件，可以提供更高的计算性能和能效。

NPU 模型编译过程较为复杂，需要配置较多的环境变量，在 NPU 编译过程中也需要本地 PC 电脑准备比较大的内存空间，并且在下载安装包时受限于网速影响，主要是使用 docker 容器进行模型编译过程。

详细的模型编译过程可参考本实训所提供的“**模型训练和编译参考文件**”文件夹中的内容自行完成模型编译的过程。

步骤 9：模型部署

模型部署就是将本地 PC 机所编译完成的模型文件导入到智能机器人中的过程，将编译后的模型文件放置于工程文件中的特定目录文件夹后，智能机器人在应用视觉功能启动垃圾巡检等程序时就会自动加载这些模型文件，就可以完成相应的垃圾识别任务了。

简单介绍一下后续一共使用了哪些模型文件，后续视觉实验中一共有三个与视觉相关的功能，分别是智能安控（sebot_safety）、人体姿态估计（sebot_pose）、垃圾巡检（sebot_polling）这三个功能类别。

1. 智能安控

智能安控在视觉方面的功能主要是用于检测人脸，引用人脸数据集以及结合 yolov3 模型完成整个训练过程，图像训练尺寸需要调整为（224×224）。

2. 人体姿态估计

人体姿态估计在视觉方面的主要功能是由于检测人体姿态，首先检测人体所在位置，继续检测人体关键点从而对人体姿态进行判断和估计。所以此功能一共用到了两个模型文件，也就是双模型结构，分别是检测模型 yolov3（图像训练尺寸为 224×224）与关键点模型 HRNet-w32（训练尺寸为 256×192）。

3. 垃圾巡检

按照本实训操作步骤即可完成垃圾巡检推理模型的训练。

上述三个功能可以进行如下总结，三个功能的共性都是使用 yolov3 模型去完成训练过程，不同之处是人体姿态估计采用了双模型，用了新模型 HRNet-w32 去训练人体关键点检

测模型。相关的数据集链接、模型训练过程、模型编译文档都以参考文档的形式进行提供，在完全理解了本次实训中的模型训练过程、相关指令功能后，就可以参考提供的文档以及数据集自行完成对应类别的模型训练任务。

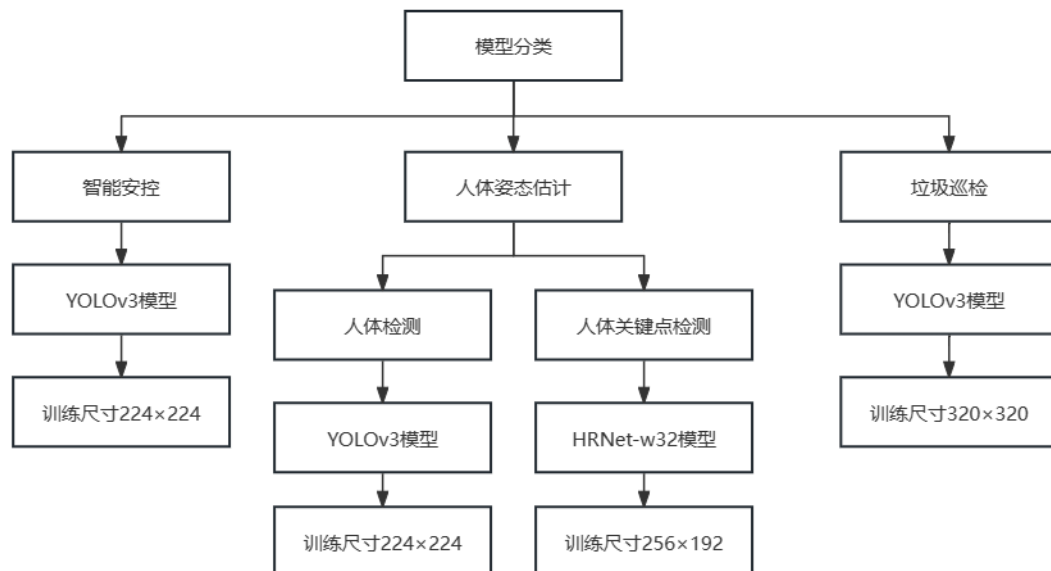


图 8-27 模型分类及训练尺寸

模型部署过程如下所示，这里我们已经完整的编译导出上述描述的这几个模型推理文件，已经提前部署至智能机器人的工程文件特定的位置。真实场景下，如果自己训练了模型并且进行编译后，通过 U 盘拷贝的方式，根据路径“sebot_t710_workspace\sebot_ros_kits\src\sebot_visions\res\models”找到工程中专用存放模型的文件夹位置将其放置进去。（注意：智能机器人中会包含这些模型文件，所以需要替换自己编译完成的模型文件，需要将这些模型文件进行备份）

其中“faceDetection”这个文件夹也是人脸检测模型，但是该模型是基于 CPU 运行的，实现效果并不理想，所以智能机器人并未使用该模型；“faceDetectionNPU”是 NPU 编译后的人脸检测推理模型；“garbageDetection”是 NPU 编译后的垃圾检测推理模型；“pedestrianDetection”是 NPU 编译后的行人检测模型，“poseEstimation”是 NPU 编译后的人体关键点检测模型。

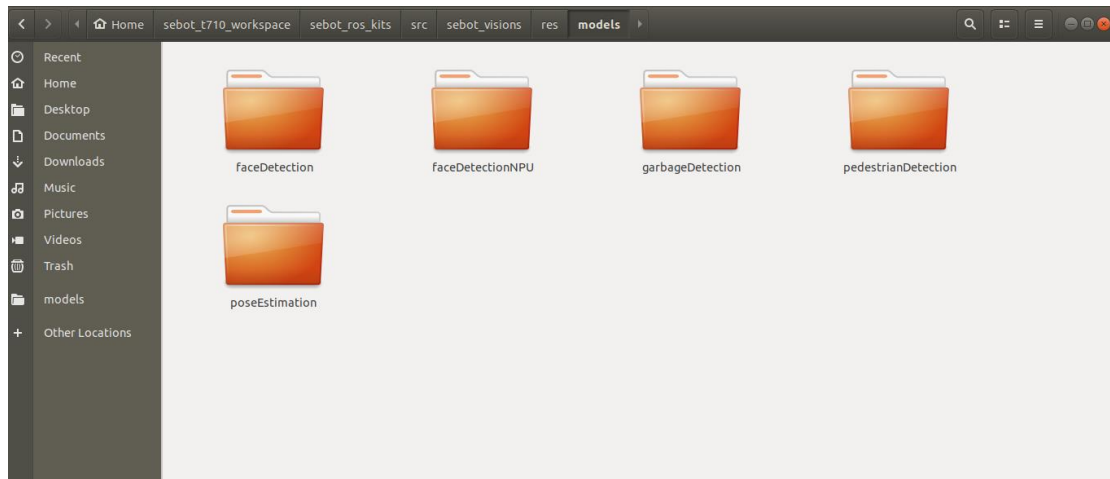


图 8-28 模型部署后的文件夹

步骤 10：练习

参照本实验所讲解的模型训练过程，根据参考文件中所提供的数据集链接，基于 AI Studio 平台完成人脸检测与行人检测的模型训练。

九、垃圾巡检与环境优化实验指导

1. 实验描述

智能机器人在酒店楼层进行工作巡逻时,可能会遇到地面有一些生活垃圾,比如说水瓶、纸团、一次性纸杯等都会被视为生活中常规垃圾。当出现个别或大量垃圾时,智能机器人会将其检测出来,根据垃圾的数量以及环境污染等级进行评分与记录,并同步做出相应的语音提示或者其他动作反馈,督促人们爱护环境。

本次课我们尝试完成垃圾巡检与环境优化实验的所有功能,会基于目标检测模型操作智能机器人进行通用垃圾检测与分类,并进行语音播报,助力酒店环境优化,掌握相关编程技术并且理解代码逻辑。

2. 实验目标

- (1) 掌握垃圾巡检与环境优化实验的逻辑与技术实现方法;
- (2) 熟悉垃圾巡检的目标检测模型应用;
- (3) 会操作智能机器人进行垃圾巡检,会对相关步骤进行操作与调试。

3. 实验步骤

步骤 1：认识垃圾巡检的工程文件

参考实训 1 的步骤,使用机器人自带的 VSCode 或通过远程连接使用个人 PC 的 VSCode 打开智能机器人中的工程文件 `sebot_t710_workspace`,在路径 “`/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/`” 找到本实验垃圾巡检的 python 工程代码文件 `sebotPolling.py`。

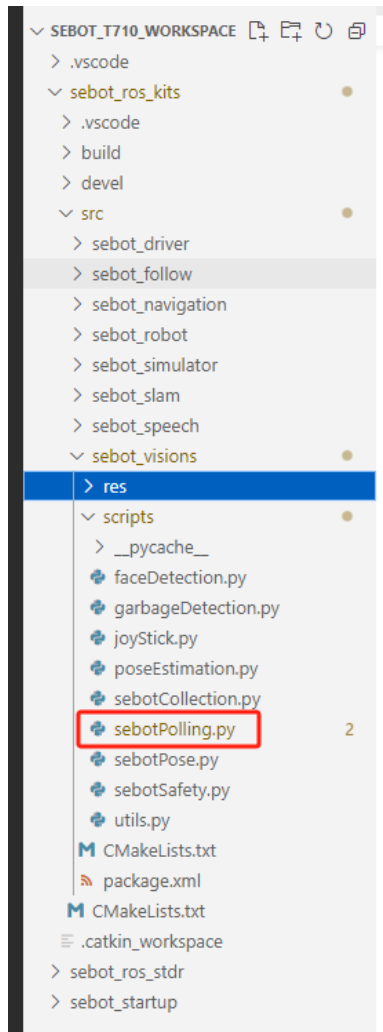


图 9-1 文件夹所在路径

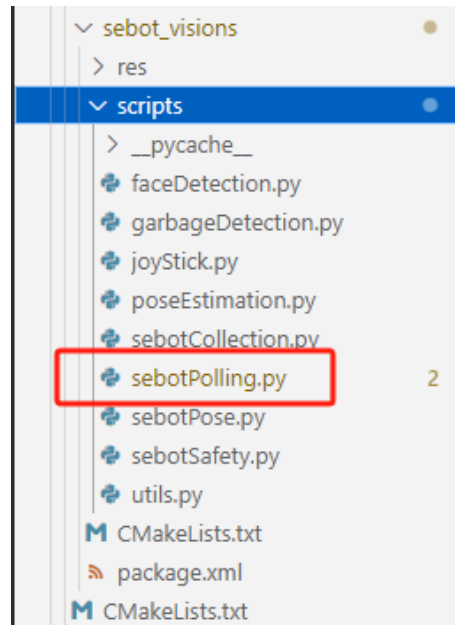


图 9-2 垃圾巡检脚本代码所在路径

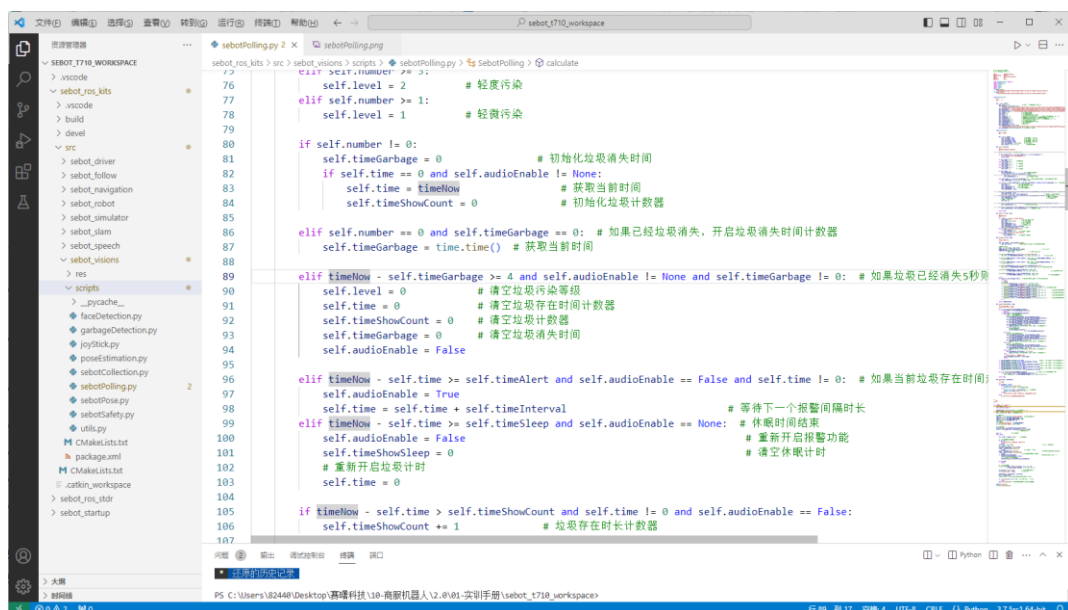


图 9-3 垃圾巡检代码文件界面

针对于这部分功能代码，接下来我们对代码的详细逻辑进行内容讲解。

步骤 2：认识机器人桌面的垃圾巡检功能界面

在步骤 1 中，我们通过个人 PC，通过远程连接的方式基于 Vscod 查看智能机器人中垃圾巡检的工程代码文件，除了这种方式，我们还可以在智能机器人系统中，通过桌面的文件夹“Files”查看智能机器人的工程文件 sebotPolling.py，路径同步骤 1。

在机器人中通过指令打开 VSCode 并启动程序后（启动指令在后续实操部分有详述），机器人系统桌面会呈现垃圾巡检的 UI 界面，如下图所示，该界面一共划分为如下 6 个区域：

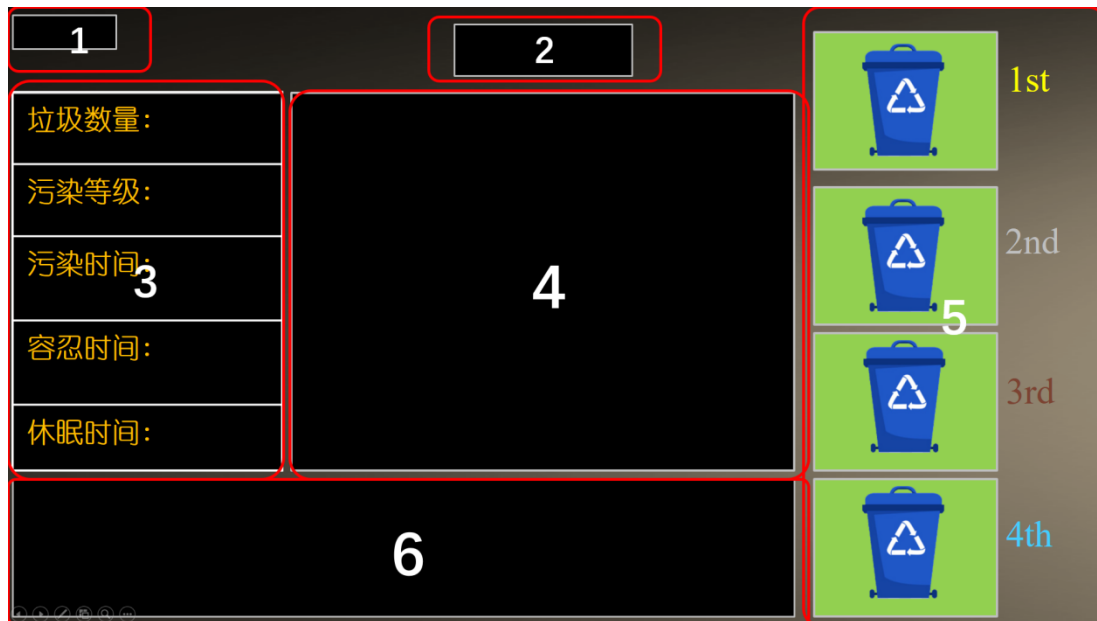


图 9-4 垃圾巡检功能界面

第一区域，表示帧率（FPS），指的是页面在一秒内连续显示的帧数。

第二区域，显示当前时间。

第三区域，显示垃圾的相关情况，一共包含垃圾数量、污染等级、污染时间、容忍时间及休眠时间这五项内容。

第四区域，显示当下摄像头采集的垃圾图像。

第五区域，对污染最严重的四个地方进行排序，并且显示排行榜每个名次的垃圾总数。

第六区域，AI 识别垃圾生成分数，如果垃圾分数达到阈值标准则显示垃圾信息展示于屏幕的最下方。

sebotPolling.py 工程代码环节将围绕着“垃圾巡检功能界面”进行数据生成，同时包含部分算法逻辑，在下一步骤进行讲解。

步骤 3：学习垃圾巡检代码逻辑

在 sebotPolling.py 文件中，代码一共包含三个主要部分：① 函数库的引入；②垃圾巡

检类，包含有垃圾巡检的各个函数功能，如函数初始化功能、垃圾图像数据功能、核算垃圾污染等级和垃圾提醒功能等等；③主程序，用于将垃圾巡检类中的各个函数功能进行拼接与逻辑梳理，完成垃圾巡检任务。

1、导入库函数（scripts/sebotPolling.py）

（1）`from garbageDetection import *`：从 `garbageDetection` 模块中导入所有内容。该模块是一个自定义的模块，其中可能包含了一些用于垃圾检测的函数和类。

（2）`import numpy as np`：导入 `numpy` 库，为了方便后续调用，将其别名设为 `np`。`numpy` 是一个用于科学计算的库，主要用于处理多维数组和矩阵。

（3）`import keyboard`：导入 `keyboard` 库，用于检测和处理键盘输入。

（4）`import time`：导入 `time` 库，用于处理时间相关的操作，例如等待一段时间或获取当前时间。

（5）`import pygame`：导入 `pygame` 库，用于创建游戏和图形界面。

（6）`import sys`：导入 `sys` 模块，用于访问和操作与 Python 解释器相关的变量和函数。

（7）`sys.path.append(...)`：将指定的路径添加到系统搜索路径中。在这段代码中，两个路径被添加到系统搜索路径中，分别是 `"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/"` 和 `"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/scripts/"`。这样做的目的是将这两个路径包含的模块和脚本添加到 Python 解释器的搜索范围内，使它们可以被导入和使用。

2、垃圾巡检类（class SebotPolling）

定义该类的目的是为了完成垃圾巡检功能，该类的定义如下：

```
class SebotPolling:
```

该类的各函数功能如下：

（1）初始化功能（`def __init__(self)`）

这部分实现了一个类的初始化函数，用于进行一些初始化操作。通过设置置信度、加载目标检测模型文件、设置音频路径、初始化计数器和变量等，为后续的垃圾检测和相关处理提供了必要的设置和准备工作。同时，读取了一张背景图片并创建了一个处理垃圾图像数据的对象。

```
def __init__(self):
    self.score = 0.25          # AI 目标检测的置信度: 0.0~1.0
    self.garbage=
    GarbageDetection("/root/workspace/sebot_t710_workspace/sebot_ros_kits
```

```

/src/sebot_visions/res/models/garbageDetection")    # 加载模型文件, 需要根据情况修改路径
    self.audioRemind=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotPolling/remind.mp3"    # 提醒音频
    self.audioClean=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotPolling/clean.mp3"    # 垃圾清理音频
    self.audioIntroduce=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotPolling/introduce.mp3"    # 引导词
    self.number = 0    # 初始化垃圾数量
    self.level = 0    # 初始化污染等级
    self.time = 0    # 初始化垃圾存在时间计数器
    self.timeAlert = 6    #报警容忍时间: timeAlert >= 6 报警容忍时间需要大于等于 6 s
    self.timeSleep = 3    #休眠时间: timeSleep > 3 休眠时间需要大于等于 3 s
    self.timeInterval = 4    #报警间隔时间: timeInterval > 4 间隔需要大于等于 4 s
    self.timeShowCount = 0    # 初始化垃圾时间计数器
    self.timeShowSleep = 0    # 初始化报警休眠时间计数器
    self.timeGarbage = 0    # 垃圾消失时间
    self.audioEnable = False    # 语音使能
    self.imgBackground=
cv2.imread("/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/images/sebotPolling.png")    # 读取背景图片
    self.garbageImg = self.GarbageImg()    # 初始化垃圾图像数据

```

(2) 定义垃圾图像数据子类

该类主要用于存储垃圾图像数据和相关信息。其属性包括垃圾数量排名前四的照片、程序计时器和垃圾数量计数器。通过创建 GarbageImg 的实例, 可以方便地记录和管理垃圾图像数据, 并实时跟踪程序的运行时间。

```

class GarbageImg:
    """
    垃圾图像数据
    """
    def __init__(self):
        self.firstImg = None    # 垃圾数量第一的照片
        self.secondImg = None    # 垃圾数量第二的照片
        self.thirdlyImg = None    # 垃圾数量第三的照片
        self.fourthlyImg = None    # 垃圾数量第四的照片

```



```
self.calculagraph = time.time()    # 程序计时器
self.counter = [0, 0, 0, 0]        # 存储垃圾数量
```

(3) 核算垃圾污染等级和垃圾提醒函数功能 (def calculate(self))

"calculate"函数主要用于核算垃圾污染等级和垃圾提醒。方法中根据一系列条件判断和计算,包括垃圾数量、时间参数等,确定垃圾的污染等级和是否触发垃圾提醒。同时,计时器和计数器被用于记录垃圾存在的时间和统计垃圾的数量。最后,根据计算结果返回 True 表示计算成功。

接下来讲解内部代码逻辑,请对照代码查阅。

在方法中,首先获取当前时间"timeNow"。然后,通过一系列条件判断计算垃圾的污染等级和垃圾提醒。

① 如果时间参数设置不正确,会打印错误信息并返回 False。

```
def calculate(self):
    """
    核算垃圾污染等级和垃圾提醒
    """
    timeNow = time.time() # 获取当前时间
    if self.timeInterval < 4 or self.timeAlert < 6 or self.timeSleep
    < 3:
        print("[Error] 时间参数设置不正确!!!")
        return False
```

②根据垃圾数量进行污染等级的判断。如果垃圾数量大于等于 10,则认为是重度污染,6 到 9 则是中度污染,3 到 5 则是轻度污染,1 到 2 则是轻微污染。

```
if self.number >= 10:
    self.level = 4    # 重度污染
elif self.number >= 6:
    self.level = 3    # 中度污染
elif self.number >= 3:
    self.level = 2    # 轻度污染
elif self.number >= 1:
    self.level = 1    # 轻微污染
```

③ 判断垃圾是否存在,一共包含有 5 个判断步骤。

如果垃圾存在,将垃圾消失时间初始化为 0。继续进行判断,如果计时器时间为 0 并且声音使能不为 None,则获取当前时间、初始化垃圾计数器。

```
if self.number != 0:
    self.timeGarbage = 0    # 初始化垃圾消失时间
    if self.time == 0 and self.audioEnable != None:
        self.time = timeNow    # 获取当前时间
```

```
self.timeShowCount = 0          # 初始化垃圾计数器
```

如果垃圾数量为 0 并且垃圾消失时间为 0，则开启垃圾消失时间计数器。

```
elif self.number == 0 and self.timeGarbage == 0: # 如果已经垃圾消失，开启垃圾消失时间计数器
```

```
self.timeGarbage = time.time() # 获取当前时间
```

如果当前时间减去垃圾消失时间大于等于 4 秒；并且声音使能不为 None；且垃圾消失时间不为 0，同时满足这三个条件时，则认为垃圾已经被清理。

将垃圾污染等级、垃圾存在时间计数器、垃圾计数器、垃圾消失时间和声音使能进行清空。

```
elif timeNow - self.timeGarbage >= 4 and self.audioEnable != None and self.timeGarbage != 0: # 如果垃圾已经消失 5 秒则认为已经被清理
    self.level = 0                # 清空垃圾污染等级
    self.time = 0                 # 清空垃圾存在时间计数器
    self.timeShowCount = 0       # 清空垃圾计数器
    self.timeGarbage = 0         # 清空垃圾消失时间
    self.audioEnable = False
```

如果当前时间减去垃圾存在时间大于等于报警容忍时间并且声音使能为 False 并且垃圾存在时间不为 0，则赋予报警使能，然后等待下一个报警间隔时长。

```
elif timeNow - self.time >= self.timeAlert and self.audioEnable == False and self.time != 0: # 如果当前垃圾存在时间超过报警容忍时间则赋予报警使能
    self.audioEnable = True
    self.time = self.time + self.timeInterval # 等待下一个报警间隔时长
```

如果当前时间减去休眠时间大于等于休眠计时器；并且声音使能为 None，同时满足这两个条件时，结束休眠，则重新开启报警功能，并清空休眠计时和垃圾计时。

```
elif timeNow - self.time >= self.timeSleep and self.audioEnable == None: # 休眠时间结束
    self.audioEnable = False                # 重新开启报警功能
    self.timeShowSleep = 0                  # 清空休眠计时
    # 重新开启垃圾计时
    self.time = 0
```

④ 计算垃圾存在时长与休眠时长。

判断当前时间减去垃圾存在时间是否大于垃圾存在时长计数器并且垃圾存在时间不为 0，且声音使能为 False，如果是，则垃圾存在时长计数器加 1。

判断当前时间减去垃圾存在时间是否大于等于休眠时间减去休眠计时器加 1，且垃圾存在时间不为 0 并且声音使能为 None，如果是，则休眠时长计数器减 1。

```
if timeNow - self.time > self.timeShowCount and self.time != 0
```

```

and self.audioEnable == False:
    self.timeShowCount += 1          # 垃圾存在时长计数器

    if timeNow - self.time > self.timeSleep - self.timeShowSleep +
1 and self.time != 0 and self.audioEnable == None:
        self.timeShowSleep = self.timeShowSleep - 1 # 休眠时长计数器

```

⑤ 结束。返回 True，表示计算完成。

```

return True

```

(4) 键盘控制阈值 (def keyControl(self, key))

"keyControl"方法，用于处理键盘输入以控制阈值参数。根据传入的按键参数，可以执行不同的操作。如果按键是"enter"，则会清空当前垃圾计数和相关变量，关闭语音使能，并播放音频。如果按键是"up"，则会增加报警所需时间的阈值。如果按键是"down"，则会减少报警所需时间的阈值。通过这些操作，可以通过键盘来控制不同的阈值设置，以适应不同的需求。

```

def keyControl(self, key):
    """
    键盘控制阈值
    """
    if str(key).__contains__("enter down"):
        self.number = 0          # 清空当前垃圾计数
        self.time = time.time()  # 清空垃圾数量
        self.level = 0           # 清空污染等级
        self.timeShowCount = 0   # 清空计时显示
        self.audioEnable = None  # 关闭语音使能
        self.timeShowSleep = self.timeSleep
        self.audio(self.audioClean) # 播放音频
    elif str(key).__contains__("up down"):
        if self.timeAlert < 15:
            self.timeAlert += 1   # 延长报警所需时间
    elif str(key).__contains__("down down"):
        if self.timeAlert > 6:
            self.timeAlert -= 1   # 缩短报警所需时间

```

(5) 显示模型预测结果 (def showResult(self, img))

"showResult"方法主要功能是用于显示模型预测的垃圾识别结果。该方法通过在图像上添加文本信息和绘制方框，将预测结果以可视化形式展示在 UI 界面上。文本信息包括当前时间、垃圾数量、污染等级、垃圾存在时间、报警容忍时间和垃圾休眠时间。通过循环遍历垃圾识别结果，将识别到的垃圾的分数和方框的边界坐标添加到图像上。最后，返回展示了识别结果的图像。这样，用户可以直观地了解模型预测的垃圾信息。

```

def showResult(self, img):
    """
    显示模型预测结果
    """
    img, self.number = self.garbage.drawBox(
        img, self.score)      # 将预测的结果绘制到图像上并获取垃圾数量
    # 调整图像大小
    img = cv2.resize(img, (880, 660))
    imgBackground = self.imgBackground.copy()    # 复制背景图
    imgBackground[150:150+660,496:496+880]=img   # 拼接图像到背景图

    cv2.putText(imgBackground, f"Time: {time.strftime('%H:%M')}", (
795, 90), cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 3,) #显示当前
时间
    cv2.putText(imgBackground, f"{self.number}", (300, 215),
cv2.FONT_HERSHEY_SIMPLEX, 1.8, (0, 255, 255), 3,)    # 显示垃圾数量
    cv2.putText(imgBackground, f"{self.level}", (300, 343),
cv2.FONT_HERSHEY_SIMPLEX, 1.8, (0, 255, 255), 3,) # 显示污染等级
    cv2.putText(imgBackground, f"{str(round(self.timeShowCount))[-
2:]}", (300,468),cv2.FONT_HERSHEY_SIMPLEX, 1.8, (0, 255, 255), 3,) #
显示垃圾存在时间
    cv2.putText(imgBackground, f"{self.timeAlert}", (300, 615),
cv2.FONT_HERSHEY_SIMPLEX, 1.8, (0, 255, 255), 3,) # 显示报警容忍时间
    cv2.putText(imgBackground, f"{self.timeShowSleep}", (300, 762),
cv2.FONT_HERSHEY_SIMPLEX, 1.8, (0, 255, 255), 3,)#显示垃圾休眠时间
    i = 0
    for result in self.garbage.result:    # 如果垃圾分数达到阈值标准则显
示垃圾信息
        if i >= 4:
            break
        # 垃圾编号
        cv2.putText(imgBackground, f"(Num:{i+1})", (18+345*i, 880),
cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 200, 200), 2,)
        cv2.putText(imgBackground,f"Score:
{str(round(result.score,3))}", (160+345*i,871),cv2.FONT_HERSHEY_SIMPLE
X, 0.8, (0, 0, 255), 2,)    # AI 识别垃圾分数
        cv2.putText(imgBackground, f"XMin: {round(result.xMin)}", (
160+345*i, 911), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (222, 222, 0), 2,)
# 方框左上角点 X 轴坐标
        cv2.putText(imgBackground, f"YMin: {round(result.yMin)}", (
160+345*i, 951), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (222, 222, 0), 2,)
# 方框左上角点 Y 轴坐标
        cv2.putText(imgBackground, f"XMax: {round(result.xMax)}", (
160+345*i, 991), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (222, 222, 0), 2,)

```

```
# 方框右下角点 X 轴坐标
    cv2.putText(imgBackground, f"YMax: {round(result.yMax)}", (
160+345*i, 1031), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (222, 222, 0), 2,)
# 方框右下角点 Y 轴坐标
    i += 1
    return imgBackground
```

(6) 显示污染最严重的四个地方 (def showGarbageImg(self, img))

"showGarbageImg"的方法主要用于显示污染最严重的四个地方, 函数功能内部的代码逻辑如下:

第一步, 通过遍历垃圾图片计数器"self.garbageImg.counter", 根据当前排行榜和当前垃圾数量进行排序。

```
def showGarbageImg(self, img):
    """
    显示污染最严重的四个地方
    """
    for i in range(len(self.garbageImg.counter)):
        if self.number == self.garbageImg.counter[i]:
            break
        if self.number > self.garbageImg.counter[i]:
            if i == 0: # 将排名向下移动
                # 将图像和垃圾数量向下移动
                self.garbageImg.fourthlyImg,
self.garbageImg.counter[
3] = self.garbageImg.thirdlyImg, self.garbageImg.counter[2]
                self.garbageImg.thirdlyImg, self.garbageImg.counter[
2] = self.garbageImg.secondImg, self.garbageImg.counter[1]
                self.garbageImg.secondImg, self.garbageImg.counter[
1] = self.garbageImg.firstImg, self.garbageImg.counter[0]
                self.garbageImg.firstImg = cv2.resize(
img[150:150+660, 496:496+880], (320, 240)) # 获取新的图像
            for j in range(3, 0):
                # 将垃圾数量更新
                self.garbageImg.counter[j+1]=
self.garbageImg.counter[j]
            elif i == 1:
                # 将图像和垃圾数量向下移动
                self.garbageImg.fourthlyImg,
self.garbageImg.counter[3]=self.garbageImg.thirdlyImg,
self.garbageImg.counter[2]
                self.garbageImg.thirdlyImg, self.garbageImg.counter[
2] = self.garbageImg.secondImg, self.garbageImg.counter[1]
                self.garbageImg.secondImg = cv2.resize(
```

```

img[150:150+660, 496:496+880], (320, 240)) # 获取新的图像
        elif i == 2:
            # 将图像和垃圾数量向下移动
            self.garbageImg.fourthlyImg,
self.garbageImg.counter[3]=self.garbageImg.thirdlyImg,
self.garbageImg.counter[2]
            self.garbageImg.thirdlyImg = cv2.resize(
img[150:150+660, 496:496+880], (320, 240)) # 获取新的图像
        elif i == 3:
            self.garbageImg.fourthlyImg = cv2.resize(
img[150:150+660, 496:496+880], (320, 240)) # 获取新的图像
        else:
            print("[Error] 照片索引出错")
            self.garbageImg.counter[i] = self.number
            break

```

第二步，通过条件判断，将包含有垃圾的图片显示在原始图像上的指定位置。如果对应包含有垃圾的图片不为空，则将图片拼接到指定的位置上，代码如下。

```

        if self.garbageImg.firstImg is not None: #如果第一名照片不为空则显示
        捕获的图像
            img[44:44+240, 1411:1411+320] = self.garbageImg.firstImg
        if self.garbageImg.secondImg is not None:#如果第二名照片不为空则显示
        捕获的图像
            img[315:315+240, 1411:1411+320] = self.garbageImg.secondImg
        if self.garbageImg.thirdlyImg is not None:#如果第三名照片不为空则显示
        捕获图像
            img[570:570+240, 1411:1411+320] =
self.garbageImg.thirdlyImg
        if self.garbageImg.fourthlyImg is not None:#如果第四名照片不为空则
        显示捕获图
            img[825:825+240, 1411:1411+320] =
self.garbageImg.fourthlyIm

```

第三步，通过循环遍历计数器，将每个名次的垃圾数量添加到图像上。

```

        for i in range(len(self.garbageImg.counter)): #显示排行榜每个名次
        的垃圾总数
            cv2.putText(img, f"Num: {self.garbageImg.counter[i]}", (
                1750, 200+i*263), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (222,
222, 0), 2,)

```

第四步，返回包含了垃圾图片和垃圾排行榜的图像。

```

return img

```

(7) 播放音频 (def audio(self, audioPath))

"audio"方法用于播放指定路径的音频文件。方法通过判断音频路径是否为空，加载音频

文件，并使用 `pygame` 库来播放音频，进行语音提醒。

```
def audio(self, audioPath):
    """
    播放音频
    """
    if audioPath != None:
        if not pygame.mixer.get_init():
            pygame.mixer.init() # 初始化声音混合器
        try:
            pygame.mixer.music.load(audioPath) # 加载音频文件
            pygame.mixer.music.play() # 开始播放
        except:
            print("[Error] 音频播放失败, 请检查音频内容及路径")
    else:
        print("[Error] 请输入正确的音频路径")
```

3、主程序（缺失代码需填空）

此部分为语音交互的主程序，会结合前文所讲述的各个类功能进行综合运用，完成垃圾巡检及语音提醒的功能任务。此部分代码可以区分为两个环节，初始化环节代码与垃圾巡检功能运行代码。

（1）初始化部分代码讲解（缺失代码需填空）

该代码片段是主程序的入口点，用于初始化并设置一些对象和参数，它导入了名为“`uart`”的模块，用于处理串口通信，以及名为“`joyStick`”的模块，用于处理遥控手柄输入，为后续的垃圾巡检和控制操作做准备。它包括初始化垃圾巡检类、串口连接对象和遥控手柄，设置摄像头参数和创建窗口，播放引导词音频，并注册键盘控制函数。这些操作为垃圾巡检和控制提供了必要的基础设置。

```
if __name__ == "__main__":
    from uart import *
    from joyStick import *

    # 初始化垃圾巡检类、串口初始化、遥控手柄初始化，共三行代码。
    # ##### 代码填空开始处 ##### #
    """
    代码填空提示：
    1. 初始化垃圾巡检类使用变量 sebotPolling 去承接
    2. 直接调用 Uart 函数，参数填写"/dev/robot"，用变量 uart 去承接
    3. 调用 JoyStick 函数，参数填写 (0, uart)，用 joy 变量去承接
    """

    # ##### 代码填空结束处 ##### #
    # 设置摄像头
```

```

capture = cv2.VideoCapture("/dev/video0") # 可以设置摄像头

capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
capture.set(cv2.CAP_PROP_FPS, 12)
# 创建窗口
cv2.namedWindow("Video", cv2.WINDOW_NORMAL)
cv2.resizeWindow("Video", 1920, 1080)

# 播放引导词
sebotPolling.audio(sebotPolling.audioIntroduce)
keyboard.hook(lambda key: sebotPolling.keyControl(key)
               ) # 开启键盘控制阈值

fps = 0 # 初始化FPS

```

(2) 垃圾巡检逻辑（缺失代码需填空）

这部分的代码实现了一个实时的垃圾巡检和控制操作的循环。通过读取摄像头的视频帧，并进行目标检测、结果显示、垃圾污染等级计算、污染地点显示、音频播放等操作。同时，通过键盘和遥控手柄可以控制机器人的运行速度。在每帧图像上绘制了帧率信息，并提供了按键退出循环的功能。循环结束后，释放资源并关闭窗口。代码详细逻辑如下：

在循环内部，首先通过“capture.read()”方法读取摄像头的视频帧。然后，检查是否成功读取视频帧，如果读取失败则打印错误信息并结束循环，之后开始正常的内部流程。

- ① 记录程序的开始时间“startTime”。
- ② 调用“sebotPolling.garbage.render()”方法进行目标检测，将垃圾检测结果渲染到帧上。
- ③ 调用“sebotPolling.showResult()”方法显示垃圾检测结果，在帧上标记垃圾的数量、污染等级等信息。
- ④ 调用“sebotPolling.calculate()”方法核算垃圾污染等级和垃圾提醒。
- ⑤ 调用“sebotPolling.showGarbageImg()”方法显示污染最严重的四个地方。如果垃圾提醒语音使能已开启，则将其关闭，并播放垃圾提醒的音频。
- ⑥ 使用“cv2.putText()”方法将帧率信息绘制在帧上，并且记录程序的结束时间“currentTime”，并计算帧率“fps”，使用“cv2.imshow()”方法显示帧的图像。
- ⑦ 如果按下“ESC”键或者“c”键，则退出循环。调用“joy.joystickHandle()”方法以获取遥控手柄的键值，用于控制机器人的速度。

⑧ 循环结束后，释放视频捕获资源，关闭窗口。

```

while True:
    ret, frame = capture.read()      # 读取视频
    # 检查是否成功读取视频帧
    if not ret:
        print("摄像头打开失败! 请检查摄像头")
        break
    startTime = time.time()          # 程序开启记录时间
    # 开启模型进行目标检测
    sebotPolling.garbage.render(frame)
    frame = sebotPolling.showResult(frame) # 显示垃圾检测结果
    # 核算垃圾污染等级和垃圾提醒
    # ##### 代码填空开始处 ##### #
    """
    代码填空提示:
    1. 调用 calculate() 函数进行计算, 并且使用变量去继承结果。
    """
    # ##### 代码填空结束处 ##### #
    frame = sebotPolling.showGarbageImg(frame)
# 显示污染最严重的四个地方
    if sebotPolling.audioEnable:      # 如果提醒语音使能已开启
        sebotPolling.audioEnable = False
        # 播放音频 提醒用户清理垃圾
        sebotPolling.audio(sebotPolling.audioRemind)

    if not type:
        break
    # 使用 cv 绘制帧率
    cv2.putText(frame, f"FPS: {round(fps, 2)}", (12, 55),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2,)

    currentTime = time.time()          # 程序关闭记录时间
    cv2.imshow("Video", frame)         # 显示图像
    elapsedTime = currentTime - startTime
    fps = 1/elapsedTime
    # print("FPS: ",str(round(elapsedTime*1000,1)), "ms")

    if cv2.waitKey(1) in [27, 99]: # 按 ESC 或 c 退出程序
        break
    joy.joystickHandle() # 遥控手柄获取键值,控制机器人速度

capture.release()
cv2.destroyAllWindows()

```

步骤 4：智能机器人垃圾巡检实操效果运行

第一步，启动程序。将机器人开机启动，打开终端，输入指令“python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotPolling.py”启动垃圾巡检程序。

```
root@paddlepi:~# python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotPolling.py
```

图 9-5 垃圾巡检程序启动

第二步，垃圾种类的识别。程序启动后，就可以在机器人的摄像头范围内呈现训练集内所包含的 5 个种类的垃圾，或者手柄遥控机器人四处走动巡逻，程序自动获取摄像头范围的垃圾图像，进行垃圾种类的识别。本程序中可检测识别的垃圾种类包括一次性纸杯、塑料瓶、果皮、纸团。同学们可操作查看 UI 界面数据变化。

当 UI 界面中摄像头的位置呈现垃圾数据时，则在页面的左侧会展示主要的五类信息。垃圾数量：数量为 1，主要就是统计摄像头画面中所呈现的垃圾个数；垃圾等级：因为数量较少，所以污染等级也比较低为 1，污染等级与垃圾数量相关；垃圾的污染时间：垃圾的存在时间已经达到 20s；报警的容忍时间：时间阈值设定为 6 秒，垃圾存在超过 6 秒钟的容忍时间则会提示报警音频；休眠时间：默认为 3 秒，可以理解是每次检测完成后会有 3 秒的冷却时间，才可以开启下次检测。UI 界面的右侧画面为垃圾数量的排行榜；UI 界面的下方为垃圾的置信度分数与所在坐标位置的排名。

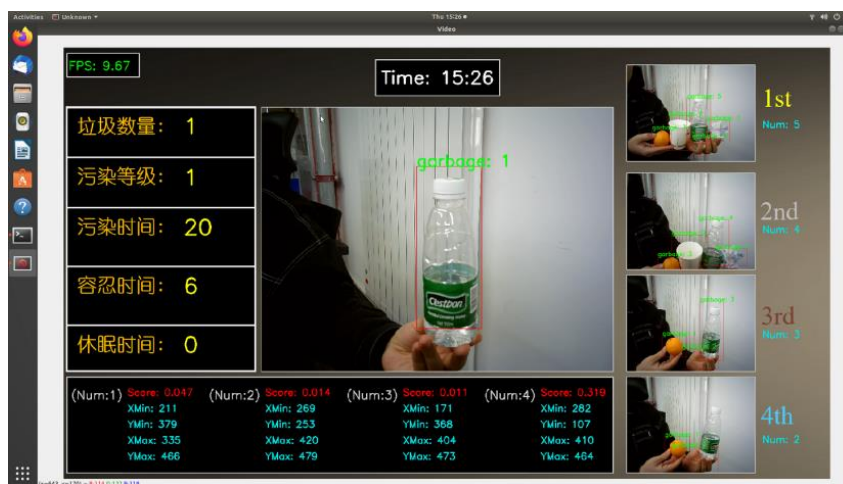


图 9-6 垃圾巡检功能

第三步，手柄控制机器人在酒店环境或实际场地中四处走动，检测环境中的垃圾，只要摄像头视野内检测到上述的一次性纸杯、塑料瓶、果皮或纸团，机器人会自动发出语音提示，发挥环境监督优化功能。

遥控手柄可以控制智能机器人四处走动、转向，只要摄像头范围内检测到垃圾，机器人会自动进行语音报警。遥控手柄操作图示如下，可以按照图中的指示操控智能机器人前进/转向。



图 9-7 手柄控制

第四步，键盘控制。根据传入的按键参数，可以控制阈值范围。

如果按下回车键，则会清空当前垃圾计数、清空垃圾数量、清空污染等级、清空计时显示，并播放音频进行提示。

如果按下方向上键，则会增加报警所需时间的阈值，阈值最大不超过 15 秒。

如果按下方向下键，则会减少报警所需时间的阈值，报警时间最小不低于 6 秒。通过这些操作，可以通过键盘来控制容忍时间的阈值设置。



图 9-8 按下回车键重置数据

最后，按 Esc（按键模式）可退出垃圾巡检程序。

十、人脸检测与智能安防实验指导

1. 实验描述

智能机器人在酒店进行巡检工作时，会遇到许许多多的行人，这些行人可能包括酒店的服务人员、保洁人员、客人等，还可能有外来的陌生人员，那么智能机器人基于其设置的摄像头可以行使智能安全监控的功能。换句话说，当智能机器人遇到行人时，可对人脸进行检测或准确识别，如果摄像头画面中存在多个人物时，也要能对多张人脸进行实时检测。

本次实验我们学习基于人脸检测模型，尝试通过源码编程实现人脸检测与智能语音播报以实现智能安全监控的功能，掌握相关编程方法与技术。

2. 实验目标

- (1) 掌握人脸检测与智能安防的 python 代码逻辑与实现方法；
- (2) 熟悉人脸检测模型应用场景；
- (3) 会操作智能机器人进行人脸检测，会对相关步骤进行操作与调试。

3. 实验步骤

步骤 1：认识人脸检测与智能安防的工程文件

启动智能机器人，打开智能机器人中的工程文件 `sebot_t710_workspace`，在路径 `“/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/”` 找到本实验智能安防的 python 工程代码文件 `sebotSafety.py`。

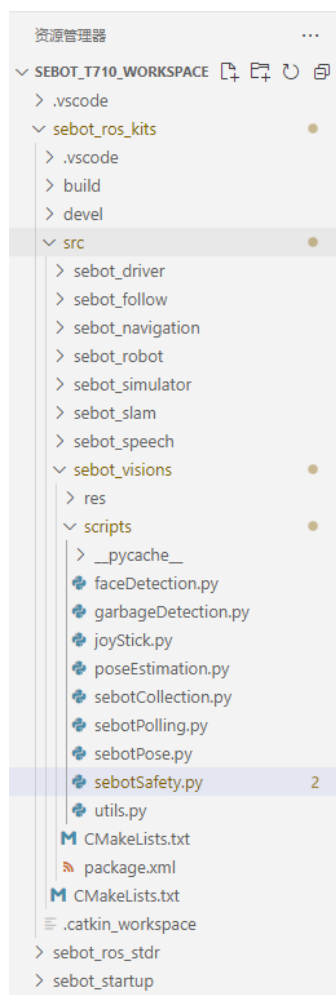


图 10-1 文件夹所在路径

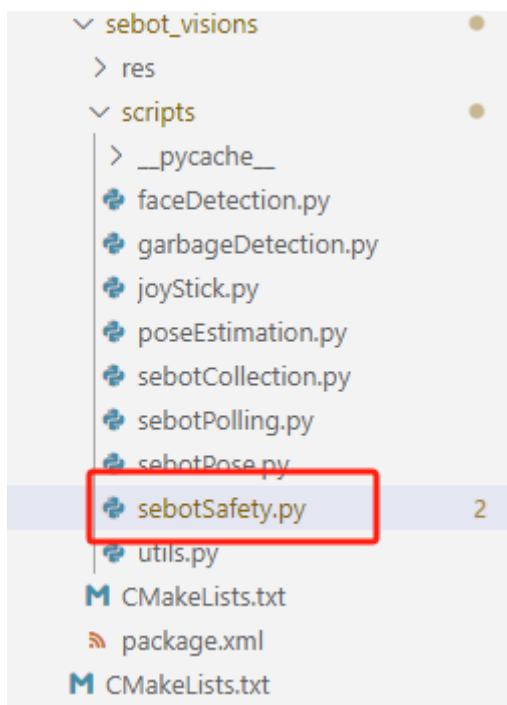


图 10-2 安控检测脚本代码所在路径

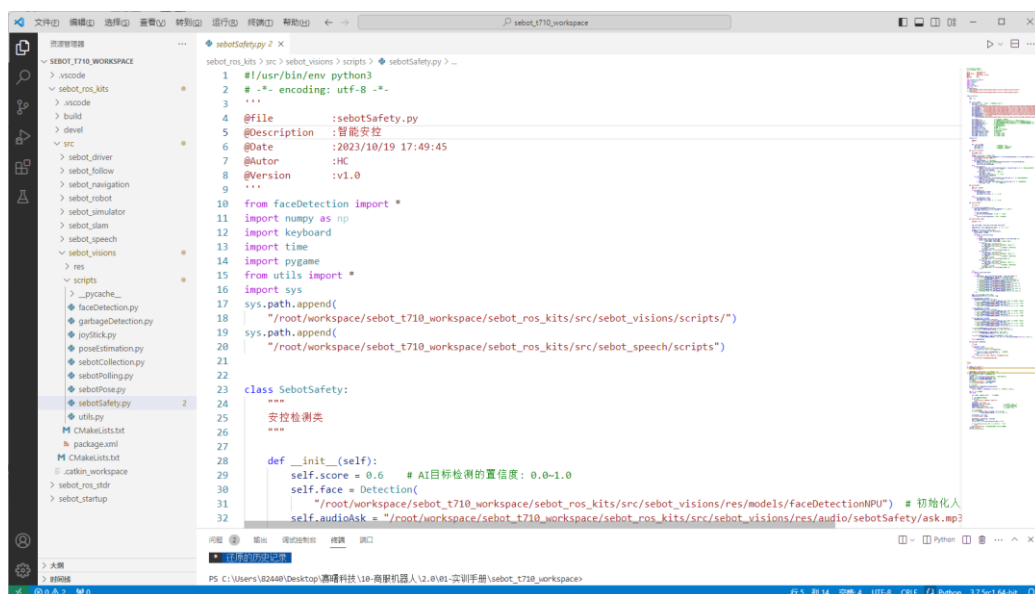


图 10-3 人脸检测与智能安控代码文件界面

针对于这部分功能代码，接下来我们对代码的详细逻辑进行内容讲解。

步骤 2：认识人脸检测与智能安控功能界面

人脸检测与智能安控的程序启动后（启动方式后续实操部分详述），机器人系统界面会呈现该功能的图形化界面展示相关数据，图形界面一共划分为如下 6 个区域，如下图所示。

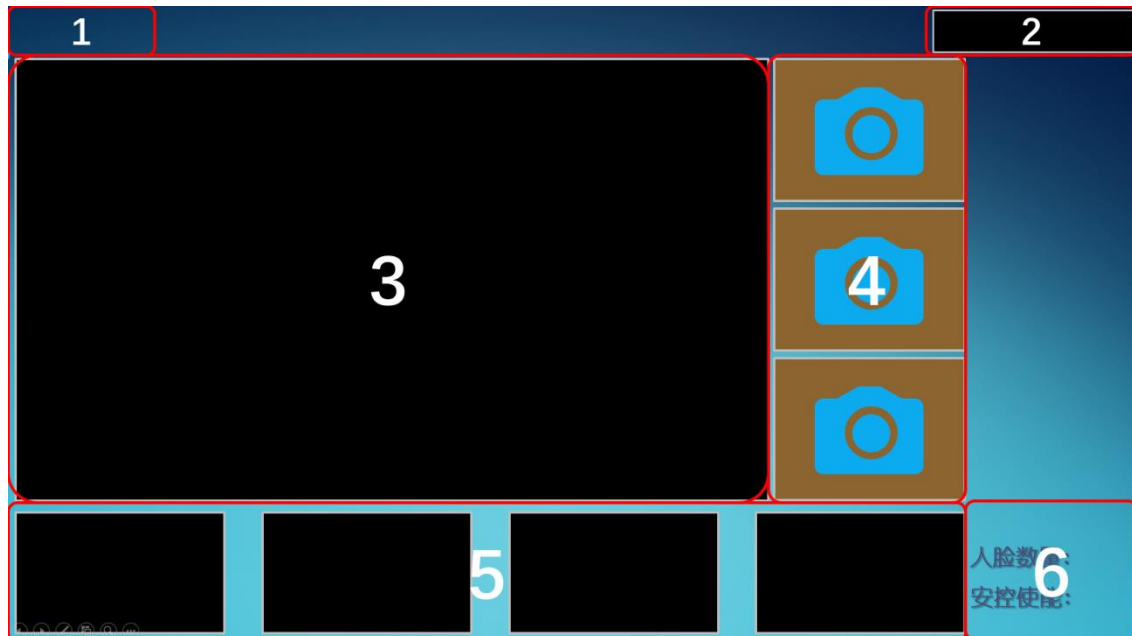


图 10-4 安控检测功能界面

第 1 区域，表示帧率（FPS），指的是页面显示在一秒内连续显示的帧数。

第 2 区域，显示当前时间。

第 3 区域，显示当前摄像头实时监测的人脸图像。

第 4 区域，拆分人脸图像，当摄像头视野中出现多张人脸时，该部分会将检测到的每张人脸进行分块展示，同时展示每张人脸检测的分数与当前的时间。

第 5 区域，对检测到的人脸得分进行排序，同步展示人脸的坐标位置。

第 6 区域，展示当前视频画面中检测人脸数量与安控是否开启的状态，包括 `True` 和 `False`。

工程代码主要围绕着“人脸检测与智能安控功能界面”进行数据生成，同时包含部分算法逻辑。

步骤 3：学习理解人脸检测与智能安控代码逻辑

`sebotSafety.py` 文件代码环节一共包含三个主要部分：① 函数库的引入；② 安控检测类，包含有安控检测的各个函数功能，有函数初始化功能、人脸数据、语音播报、显示人脸检测结果等功能等；③ 主程序，用于将安控检测类的各个函数功能进行结合，最终完成智能机器

人智能安控的任务。

1、导入库函数（scripts/sebotSafety.py）

- （1）`from faceDetection import *`: 从"faceDetection"模块中导入所有的函数和变量。
- （2）`import numpy as np`: 导入 numpy 模块并将其命名为 np，用于进行数值计算和数组操作。
- （3）`import keyboard`: 导入 keyboard 模块，用于检测和响应键盘按键事件。
- （4）`import time`: 导入 time 模块，用于处理时间相关的操作。
- （5）`import pygame`: 导入 pygame 模块，用于创建基于图形的用户界面和游戏。
- （6）`from utils import *`: 从"utils"模块中导入所有的函数和变量。
- （7）`import sys`: 导入 sys 模块，用于访问和操作与 Python 解释器相关的变量和函数。
- （8）`sys.path.append("/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/")`: 将指定路径添加到 Python 解释器搜索模块的路径列表中，以便在该路径下查找导入的模块。
- （9）`sys.path.append("/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/scripts/")`: 同上，将另一个路径添加到模块的搜索路径列表中。

```
from faceDetection import *
import numpy as np
import keyboard
import time
import pygame
from utils import *
import sys

sys.path.append("/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/")
sys.path.append("/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_speech/scripts/")
```

这段代码主要是在 Python 中导入了一些模块和函数库，包括用于人脸检测的模块、数值计算和数组操作的模块、键盘事件响应的模块、时间处理模块、图形界面和游戏的模块以及一些自定义的工具函数模块。通过导入这些模块和函数，可以在后续代码中使用它们提供的功能和工具，使得编程更加高效和方便。

2、安控检测类（class SebotSafety）

定义该类的目的是为了完成安控检测功能，该类的定义如下：


```
class SebotSafety:
```

该类的各函数功能如下：

(1) 初始化功能（def __init__(self)）

这段代码是一个类的构造函数，用于初始化类的属性和变量。它设置了目标检测的置信度阈值和人脸检测对象，并指定了一些音频文件路径、背景图片路径以及时间间隔、开关状态等属性的初始值，还加载了人脸检测模型用于完成人脸检测功能，构造函数还创建了三个(A、B、C)图像对象，并调用了 **Result()**方法进行初始化。综合而言，该代码段为类的实例化提供了必要的参数和默认值，用于后续的人脸检测和图像处理操作。在源文件中有相关代码的注释，可详看。

```
def __init__(self):
    self.score = 0.6    # AI 目标检测的置信度： 0.0~1.0
    self.face = Detection(
        "/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/models/faceDetectionNPU") # 初始化人脸检测
    self.audioAsk=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotSafety/ask.mp3"    # 无人询问音频
    self.audioOpen=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotSafety/open.mp3"    # 开启安控音频
    self.audioClose=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotSafety/close.mp3"    # 关闭安控音频
    self.audioFun=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotSafety/fun.mp3"    # 功能介绍音频
    self.audioGreet=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotSafety/greet.mp3"    # 问候音频
    self.audioIntroduce=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotSafety/introduce.mp3" # 引导词
    self.audioInvade=
"/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/audio/sebotSafety/invade.mp3"    # 入侵提示音频
    self.imgBackground = cv2.imread(
        "/root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/res/images/sebotSafety.png") # 读取背景图片

    self.number = 0    #初始化人脸检测的数量
```

```

        self.timeInterval = 5#设置问候间隔时间 timeSearch >= 2 问候时间
        需要大于等于 2 s
        self.timeSearch = 10 #设置寻人问候的等待时间 timeSearch >= 4 寻
        人问候时间需要大于等于 4 s
        self.timeintroduction = 5 # 设置自我介绍时间
        timeintroduction >= 5 自我介绍时间需要大于等于 5 s
        self.timeSecurity = 2 # 设置安控触发时间
        self.faceInterval = 2 # 设置人脸更新间隔
        self.greet = False # 问候使能
        self.detection = False # 人脸检测使能
        self.time = time.time() # 已检测/未检测的时间
        self.timeFace = time.time() # 人脸更新时间
        self.securityEnable = False # 安控使能
        self.imgA = self.Result() # 初始化 A 图像信息
        self.imgB = self.Result() # 初始化 B 图像信息
        self.imgC = self.Result() # 初始化 C 图像信息

```

(2) 定义图像数据子类 (class Result)

定义了一个名为 **Result** 的子类，用于存储人脸检测的结果。它包含了一个图像属性、一个时间属性和一个分数属性，用于储存相应的信息。这个类可以在人脸检测过程中创建并更新实例，存储相关数据。

```

class Result:
    """
    人脸数据
    """
    def __init__(self):
        self.img = None # 储存人脸检测结果
        self.time = 0 # 储存人脸检测的时间信息
        self.score = 0 # 储存人脸检测的分数

```

(3) 机器人问候和语音介绍的函数功能 (def introduction(self))

机器人针对检测的人脸，可以发出问候的语音，或语音介绍相关内容。此处定义一个名为 **introduction** 的方法，用于控制机器人的语音问候和介绍行为。核心逻辑如下：

- 1) 获取当前时间。
- 2) 检查一些时间属性的值是否小于指定的阈值，如果小于则输出错误信息并返回。
- 3) 如果安控使能开启。① 检查当前时间与最后一次检测时间之间的时间间隔是否超过安控触发所设定的时间；② 最后一次检测时间不为零；③检测状态开启。如果同时满足这三个条件，则更新最后一次检测时间为当前时间，并播放陌生人入侵提示音频，行使安全监控功能。

```

def introduction(self):

```

```

"""
机器人问候和介绍
"""

timeNow = time.time() # 获取当前时间
if self.timeInterval < 2 or self.timeSearch < 4 or
self.timeintroduction < 5 or self.timeSecurity < 2:
    print("[Error] 时间不能为负数!!!")
    return False

if self.securityEnable: # 进入安控模式
    if timeNow - self.time > self.timeSecurity and self.time !=
0 and self.detection:
        self.time = time.time() # 开启检测计时
        self.audio(self.audioInvade)
    else:
        if self.detection:
            if timeNow - self.time > self.timeintroduction + 4 and
self.time != 0: # 计时到自我介绍时间
                self.time = time.time() # 开启检测计时
                self.audio(self.audioFun)
            if self.greet: # 开启问候使能
                self.greet = False
                self.time = time.time() # 开启检测计时
                self.audio(self.audioGreet)
            elif not self.detection:
                if timeNow - self.time > self.timeSearch and self.time !=
0: # 计时到寻人问候时间
                    self.time = time.time() # 开启检测计时
                    self.audio(self.audioAsk)
                if timeNow - self.time > self.timeInterval and
self.time != 0: # 计时到问候时间
                    self.greet = True # 开启问候使能

```

(4) 判断是否检测到人脸 (decide(self))

方法逻辑如下：

1) 如果人脸检测数量不为零（表示检测到了人脸），并且当前的人脸检测状态为关闭，则将状态更新为开启，并记录当前时间作为最后一次检测时间。

2) 如果人脸检测数量为零（表示没有检测到人脸），并且当前的人脸检测状态为开启，则将状态更改为关闭，并记录当前时间作为最后一次检测时间。

```

def decide(self):
    """
    判断是否检测到人脸
    """

```

```

if self.number != 0:
    if self.detection == False:
        self.detection = True
        self.time = time.time() # 开启检测计时
    else:
        if self.detection == True:
            self.detection = False
            self.time = time.time() # 开启检测计时

```

(5) 反转安控使能 (def reversal(self))

该方法用于反转安控使能状态，并提供相应的音频反馈。通过改变安控使能状态，该方法可以控制安控功能的开启和关闭，并根据状态的改变提供不同的音频提示。方法逻辑如下：

1) 检查安控使能属性的数据类型，如果是布尔类型，则反转安控使能的状态，即从开启变为关闭，或从关闭变为开启，并且更新最后一次检测时间为当前时间。

2) 如果安控使能状态为开启，则播放安控检测开启音频；如果安控使能状态为关闭，则播放安控检测关闭音频。

```

def reversal(self):
    """
    反转安控使能
    """
    if type(self.securityEnable) == bool:
        self.securityEnable = not self.securityEnable # 反转安控使能
        self.time = time.time() # 开启检测计时
        if self.securityEnable:
            self.audio(self.audioOpen) # 安控检测开启音频
        else:
            self.audio(self.audioClose) # 安控检测关闭音频

```

(6) 显示人脸检测结果 (def showFace(self, img))

该方法综合利用人脸检测结果和相关信息，将其显示在图形化界面上供用户查看。除了显示人脸框、编号、分数和坐标，还显示了检测时间、人脸数量和安控使能状态等信息。返回的背景图经过绘制和更新后，用于在屏幕上展示图像及相关信息。

```

def showFace(self, img):
    """
    显示人脸检测结果
    """

```

代码较多，我们分段进行拆解，完整代码请查看源代码。方法的逻辑如下：

第一步，绘制人脸检测框并计算人脸数量。

```

img, self.number = self.face.drawBox(img, self.score)

```

第二步，复制背景图以便后续的绘制。

```
imgBackground = self.imgBackground.copy() # 复制一份背景图
```

第三步，获取当前时间并检查是否满足显示人脸检测结果的时间间隔。如果满足条件，则遍历人脸检测结果并更新人脸检测的结果图像、人脸检测的时间信息和人脸检测的分数信息等。

```
timeNow = time.time() # 获取当前时间
if timeNow - self.timeFace > self.faceInterval:
    self.timeFace = timeNow
    i = 0
    for result in self.face.result:
        if i >= 4:
            break
        if result.score > self.score and result.yMin > 0 and result.xMin > 0:
            coord = [result.xMin, result.yMin,
                    result.xMax, result.yMax] # 人脸图像的坐标
            if i == 0:
                self.imgA.img = cutImageFromDet(
                    img, coord, [320, 240]) # 储存人脸检测的结果图像
                self.imgA.time = time.strftime(
                    '%H:%M') # 储存人脸检测的时间信息
                # 储存人脸检测的分数
                self.imgA.score = str(round(result.score, 3))
            elif i == 1:
                self.imgB.img = cutImageFromDet(
                    img, coord, [320, 240]) # 储存人脸检测的结果图像
                self.imgB.time = time.strftime(
                    '%H:%M') # 储存人脸检测的时间信息
                # 储存人脸检测的分数
                self.imgB.score = str(round(result.score, 3))
            elif i == 2:
                self.imgC.img = cutImageFromDet(
                    img, coord, [320, 240]) # 储存人脸检测的结果图像
                self.imgC.time=time.strftime('%H:%M')
# 储存人脸检测的时间信息
# 储存人脸检测的分数
                self.imgC.score = str(round(result.score, 3))
            i += 1
```

第四步，通过遍历人脸检测结果列表，从中选择分数较高的前 4 个人脸，并在背景图上绘制这些人脸的编号、分数和坐标信息。通过设置计数器的方式，确保最多绘制 4 个人脸的信息。这些绘制的文本信息包括人脸编号、分数、左上角和右下角的坐标等。将分数、坐标

位置相关信息绘制在背景图像下方的对应位置（前面介绍的图形化界面的下方四个位置）。

```
i = 0
for result in self.face.result:
    if i >= 4:
        break
    if result.score > self.score and result.yMin > 0 and result.xMin > 0:
        cv2.putText(imgBackground, f"(Num:{i+1})", (20+420*i, 90
0),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (200, 200, 200), 2,)
        cv2.putText(imgBackground, f"Score: {str(round(result.sc
ore,3))}", (160+420*i, 891), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 25
5), 2,)
        cv2.putText(imgBackground, f"XMin: {round(result.xMin)}
", (160+420*i, 931), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2,)
        cv2.putText(imgBackground, f"YMin: {round(result.yMin)}
", (160+420*i, 971), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2,)
        cv2.putText(imgBackground, f"XMax: {round(result.xMax)}
", (160+420*i, 1011), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255),
2,)
        cv2.putText(imgBackground, f"YMax: {round(result.yMax)}
", (160+420*i, 1051), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255),
2,)
    i += 1
```

第五步，将输入图像缩放并覆盖到图形化界面的特定位置。

```
img = cv2.resize(img, (1280, 750))
imgBackground[90:90+750, 13:13+1280] = img
```

第六步，如果存在人脸检测结果 A，则在背景图上显示 A 的图像、时间和分数信息；如果存在人脸检测结果 B，则在背景图上显示 B 的图像、时间和分数信息；如果存在人脸检测结果 C，则在背景图上显示 C 的图像、时间和分数信息。

```
if self.imgA.img is not None:
    imgBackground[92:92+240, 1304:1304 +320] = cv2.resize(self.
imgA.img, (320, 240)) # 显示人脸检测的结果
    cv2.putText(imgBackground, f"Time: {self.imgA.time}", (
1640, 150), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 0, 255), 2,) # 显示检测
时间
    cv2.putText(imgBackground, f"Score: {self.imgA.score}", (
1640, 250), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0), 2,) # 显示检测
分数
if self.imgB.img is not None:
    imgBackground[345:345+240, 1304:1304 + 320] = cv2.resize(sel
f.imgB.img, (320, 240)) # 显示人脸检测的结果
```

```

        cv2.putText(imgBackground, f"Time: {self.imgB.time}", (
1640, 400), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 0, 255), 2,) # 显示检测
时间

        cv2.putText(imgBackground, f"Score: {self.imgB.score}", (
1640, 500), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0), 2,) # 显示检测
分数

        if self.imgC.img is not None:
            imgBackground[599:599+240, 1304:1304 + 320] = cv2.resize(sel
f.imgC.img, (320, 240)) # 显示人脸检测的结果
            cv2.putText(imgBackground, f"Time: {self.imgC.time}", (
1640, 650), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 0, 255), 2,) #显示检测
时间

            cv2.putText(imgBackground, f"Score: {self.imgC.score}", (
1640, 750), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0), 2,) # 显示检测
分数

```

第七步，在背景图上显示当前时间、人脸数量和安控使能状态。

```

        cv2.putText(imgBackground, f"Time: {time.strftime('%H:%M')}", (
1575, 60), cv2.FONT_HERSHEY_SIMPLEX, 1.8, (255, 255, 255),
2,)

        cv2.putText(imgBackground, f"{self.number}", (1825, 945),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2,)
        cv2.putText(imgBackground, f"{self.securityEnable}", (1825, 101
5),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0) if self.secu
rityEnable else (0, 0, 255), 2,)

```

最后，返回背景图。

```

return imgBackground

```

(7) 播放音频 (def audio(self, audioPath))

"audio"方法用于播放指定路径的音频文件。它首先检查音频路径是否为空，并初始化声音混合器（如果未初始化），然后尝试加载指定的音频文件并开始播放，如果出现异常则输出错误消息。

```

def audio(self, audioPath):
    """
    播放音频
    """
    if audioPath != None:
        if not pygame.mixer.get_init():
            pygame.mixer.init() # 初始化声音混合器
        try:
            pygame.mixer.music.load(audioPath) # 加载音频文件
            pygame.mixer.music.play() # 开始播放

```

```

except:
    print("[Error] 音频播放失败, 请检查音频内容及路径")
else:
    print("[Error] 请输入正确的音频路径")

```

3、主程序（缺失代码需填空）

此部分为安控检测的主程序，会结合前文所讲述的各个类功能进行综合运用，完成智能安控的功能任务。此部分代码可以区分为两个环节，初始化环节代码与智能安控功能运行代码。

（1）初始化部分代码讲解（代码填空）

该代码片段是主程序的入口点，用于初始化并设置一些对象和参数，它导入了名为“uart”的模块，用于处理串口通信，以及名为“joyStick”的模块，用于处理遥控手柄输入，为后续的智能机器人控制操作做准备。它包括初始化安控检测类、串口连接对象和遥控手柄，设置摄像头参数和创建窗口，播放引导词音频，并提示按下“Enter”按键可以控制安控使能的状态。这些操作为人脸检测与智能安控提供了必要的基础设置。

```

if __name__ == "__main__":
    from uart import *
    from joyStick import *
    # 初始化安控检测类、串口初始化、遥控手柄初始化，共三行代码。
    # ##### 代码填空开始处 ##### #
    """
        代码填空提示：
        1. 初始化安控检测类使用变量 SebotSafety 去承接
        2. 直接调用 Uart 函数，参数填写"/dev/robot", 用变量 uart 去承接
        3. 调用 JoyStick 函数，参数填写 (0, uart)，用 joy 变量去承接
    """
    # ##### 代码填空结束处 ##### #
    # 设置摄像头
    capture = cv2.VideoCapture("/dev/video0") # 可以设置摄像头
    capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    capture.set(cv2.CAP_PROP_FPS, 12)
    # 创建窗口
    cv2.namedWindow("Video", cv2.WINDOW_NORMAL)
    cv2.resizeWindow("Video", 1920, 1080)
    # 播放引导词
    sebotSafety.audio(sebotSafety.audioIntroduce)
    keyboard.on_press_key(
        "enter", lambda _: sebotSafety.reversal()) #Enter 键按下反转安控使能

```



```
fps = 0 # 初始化帧率
```

(2) 安控检测逻辑（代码填空）

第二部分的代码主要实现了实时的人脸检测与智能安控操作的循环,用于读取视频帧并进行人脸检测、安全判断以及机器人问候和语音介绍的处理。在每一帧上,它显示人脸检测结果和帧率,并通过手柄控制函数来控制机器人的速度。该循环的目的是实时处理视频流,提供相关功能和反馈,并且在接收到退出指令时退出程序。代码逻辑如下:

- 1) 在一个无限循环中,不断读取视频的帧;
- 2) 检查是否成功读取视频帧。如果读取失败,输出错误消息并跳出循环;
- 3) 记录当前时间作为起始时间;
- 4) 执行人脸目标检测,使用 `sebotSafety.face.render` 方法处理当前帧;
- 5) 判断是否检测到人脸,使用 `sebotSafety.decide` 方法根据人脸检测结果更新安全状态;
- 6) 进行机器人的问候和介绍,使用 `sebotSafety.introduction` 方法执行相关逻辑;
- 7) 将当前帧作为参数传递给 `sebotSafety.showFace` 方法,显示人脸检测结果;
- 8) 使用 OpenCV 绘制帧率信息;
- 9) 获取当前时间并计算帧率;
- 10) 将处理后的帧显示在窗口中;
- 11) 如果按下 ESC 键或 C 键,跳出循环,退出程序;
- 12) 执行手柄控制函数,用于获取遥控手柄的键值并控制机器人速度;
- 13) 释放视频的捕获对象和关闭窗口。

```
while True:
    ret, frame = capture.read()          # 读取视频
    # 检查是否成功读取视频帧
    if not ret:
        print("摄像头打开失败! 请检查摄像头")
        break
    startTime = time.time()
    sebotSafety.face.render(frame)        # 开启模型进行目标检测
    sebotSafety.decide()                  # 判断是否检测到人脸
    sebotSafety.introduction()            # 开启机器人问候和介绍
    frame = sebotSafety.showFace(frame)   # 显示人脸检测结果

    # 使用 cv 绘制帧率
    cv2.putText(frame, f"FPS: {round(fps, 2)}", (10, 30),
                  cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2,)
```

```

currentTime = time.time()
cv2.imshow("Video", frame)
# ##### 代码填空开始处 ##### #
"""
    代码填空提示：获取当前时间并计算帧率
    1. 经过时间=当前时间-开始时间
    2. fps = 1/经过时间
"""
# ##### 代码填空结束处 ##### #
# print("FPS: ",str(round(elapsedTime*1000,1)), "ms")
if cv2.waitKey(1) in [27, 99]: # 按 ESC 或 c 退出程序
    break
joy.joystickHandle() # 遥控手柄获取键值,控制机器人速度
capture.release()
cv2.destroyAllWindows()

```

步骤 4：实操运行人脸检测与智能安控程序，观测效果

第一步，将机器人开机启动，打开终端，输入指令“python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotSafety.py”启动安控检测功能。

```

root@paddlepi:~# python3 /root/workspace/sebot_t710_workspace/sebot_ros_kits/src/sebot_visions/scripts/sebotSafety.py

```

图 10-5 安控检测功能启动

第二步，安控程序启动。程序默认的是智能安控使能关闭，此时安控功能还没有开启，当摄像头视野出现人脸时，机器人会自动检测多张人脸信息并发出欢迎语或自行进行机器人自我介绍，而不是发出警报。

如下图所示，我们可以看到人脸数量显示的是 2，安控使能显示的结果为“False”（代表的是关闭状态），右侧展示部分将人脸分割出来，并且进行时间和分数的标记；UI 界面的底部，显示相应的评分排名与人脸坐标位置。

如果检测到人脸后，机器人会进行语音问候，每间隔 5 s 进行一次；连续检测超过 5 s 则进行一次自我介绍；如果 10 s 没有检测到人脸，机器人会自动发问“人都去哪儿了呢”等语音。具体时间频率设置可参考之前的代码文件对应位置，语音内容是录制好的音频文件。



图 10-6 未启动安控使能时

第三步，智能安控启动，发挥报警功能。按下键盘中的“Enter”按键则启动安控使能，此时不会再有上一步的问候或自我介绍语音，而是启动语音报警功能。此时若检测到人脸，则会出现语音报警的相应内容，如下图，右下角显示人脸数量为 1，且安控使能显示为“True”，在安防开启模式下每 2 s 对人脸进行警报。应用于酒店场景下，即发现人脸代表有人入侵，则会发出警示，继而引发干预。



图 10-7 启动安控使能时

第四步，手柄控制机器人在酒店环境或实际场地中四处走动，检测人脸，发挥安全监控

作用。遥控手柄可以控制智能机器人四处走动或转向，只要摄像头范围内检测到人脸，机器人会自动进行语音报警。遥控手柄按键操作在前面实验已讲，此处不赘述。

第五步，键盘控制智能安控功能的开启。这一步主要涉及的是安控使能的开启/关闭，主要使用“Enter”按键。

最后，按下 ESC 即可正常退出程序。